What a Loop Buys: Parameter Efficiency on Text, Extendable Depth on a Serial Task

Qirun Li *qli858246@gmail.com*
Independent researcher

Abstract

When does running a block of layers again work as test-time compute? A looped transformer adds depth without adding parameters, and two benefits are claimed: it behaves like a larger model, and more loops at inference solve harder problems without a written chain of thought. We test both, on web text in a pre-registered study (111 models of 10M–80M parameters, dense to 160M, and extensions) and on a synthetic serial task whose design was frozen before its main runs. None of the four pre-registered hypotheses held as stated, and the pre-registered effective-parameter exponent is not identified. In an exploratory analysis without a fitted law, each parameter of a model whose middle block is looped 4 times is worth at most about one dense parameter at 10–20M with up to 20 training tokens per parameter and about 1.3–1.5 at 40M–160M with 20–40, rising with data; at 40M a dense model of the same size and nearly the same depth recovers about half of this advantage, but not the part that grows with data. At equal training compute and a shared learning rate the dense model wins every measured comparison, and extra loops at inference never help on text. On chains of table lookups, the same architecture computes the chain step by step inside the loop. Within the training range this buys depth, but a chain of thought trained on every intermediate value is perfect there at under a third of the 8-loop model's compute. Beyond the range only the loop extends: a model trained with 8 loops on chains of at most 8 steps goes from 0.04 to 0.83 accuracy on 12-step chains with 16 loops, and for most training regimes its own confidence tells it when to stop; the chain of thought, whose prompt grows with the chain, does not extend. At equal loss, a middle block looped 4 times saves at most 19% of weight memory but costs 2.2–2.5 times the KV cache and decoding time. Code, logs and results are released.

1 Introduction

A looped, or recurrent-depth, transformer applies one block of layers several times in a row (Dehghani et al., 2019; Geiping et al., 2025; Zhu et al., 2025). Running a block r times costs r times its compute and adds no parameters. Two benefits are expected in return. The first is *parameter efficiency*: a looped model should behave like a larger dense model, which matters wherever memory or bandwidth rather than arithmetic limits deployment. The second is *latent reasoning*: if the loop iterates on a problem, running more loops at inference should solve harder problems without writing out a chain of thought, a form of test-time compute that lives inside the network.

Both claims are hard to check from the literature. Studies differ in data, tokenizer, training recipe and model family; seeds are rarely reported; and the quantity of interest, how much one more loop is worth, is usually read off a single fitted scaling law (Schwethelm et al., 2026; Prairie et al., 2026; Chen et al., 2026a). Evidence for test-time scaling comes from two directions that are seldom put side by side: large models trained with a random number of loops, which improve on reasoning benchmarks as more loops are run at inference, up to a task-dependent saturation (Geiping et al., 2025), and small models on synthetic tasks, which solve longer problems when run longer (Fan et al., 2025; Kohli et al., 2026; Zhang et al., 2026).

We test both claims in one study, with one architecture family and one recipe, and every plan was frozen before its runs. On web text, a pre-registered grid of 111 runs varies model size (10M–80M looped, dense

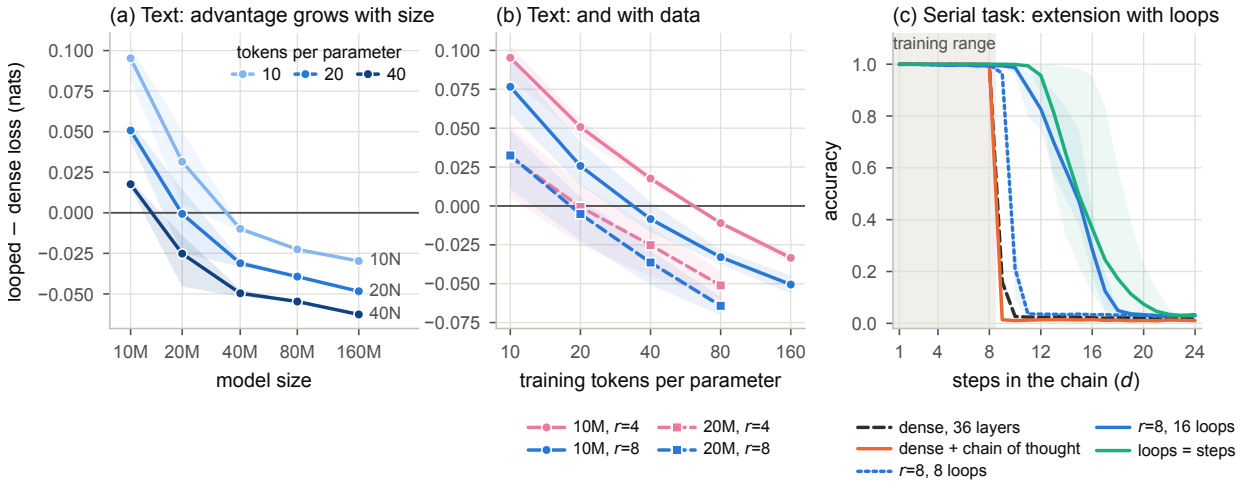


Figure 1: What a loop buys, in one figure. **(a)** On web text, at equal parameters and tokens, a middle block looped 4 times goes from worse than the dense model at 10M to better from 40M on (10, 20 and 40 tokens per parameter; band: range over seeds). **(b)** The advantage grows with the tokens per parameter (10M and 20M, 4 and 8 loops). **(c)** On chains of table lookups, trained on 1–8 steps (shaded), a dense model of the executed depth of 8 loops (36 layers) and a dense model that writes the chain out, whose prompt grows with the chain, stop at the training range; looped models run longer solve longer chains (one trained with 8 loops and run with 16, one trained with as many loops as steps; mean over five seeds, band: range).

references to 160M), which layers are looped, how often and how they are backpropagated, and extensions add size, data, re-tuned learning rates and dense models of nearly the looped model's depth. When text left the second claim untested, we built a task that needs serial steps, froze its design and predictions (replacing its modular arithmetic by chains of lookups in fixed random tables for learnability before any formal run), and compared looped models with dense models of equal parameters and of equal executed depth, with a written chain of thought and with pause tokens.

The two settings give different answers (Figure 1, Table 1). On text the loop buys parameter and data efficiency, not compute efficiency: from about 40M on, a model whose middle block is looped 4 times is worth more than its parameters, the more so with more data, and at 40M a dense model of the same size and nearly the same depth recovers about half of that advantage; at equal training compute and a shared learning rate the dense model wins every measured comparison, and extra loops at inference never help: depending on its training, the looped block runs a fixed-length program, a cycle or a fixed point. On the chain task the same architecture computes the chain step by step inside the loop. Within the training range this buys depth, which a written chain of thought buys more cheaply; beyond it only the loop extends, and for most training regimes its own confidence tells it when to stop.

We read the difference as a matter of three conditions, which the chain task meets once the model is trained for them: *headroom* (errors come from too few serial steps, not from missing knowledge), *extendable iteration* (the loop can run longer without destroying its state) and a *stopping signal*. On web text at these sizes we saw no sign of the first. This is an interpretation of two settings, not a tested law; Section 7 says what would test it.

Section 3 gives the setup. Sections 4 and 5 test the two claims on text and Section 6 on the chain task; Section 7 interprets the difference, and Section 8 states the limitations and the record behind every number: the pre-registration, a dated log of every deviation and correction, and the results and training logs of all 242 language-model runs and 135 synthetic-task runs.

Table 1: The two claims on the two tasks, by what the looped model is compared with. Looped models run with their training loop count unless stated; numbers are for a middle block looped 4 times on text.

Compared with	Web text, 10M–160M	Lookup chains, 6.6M, trained on 1–8 steps
Claim one: a looped model behaves like a larger one		
Dense, equal parameters and data	At 10–40 tokens per parameter: worse at 10M, mixed at 20M, better from 40M on, worth 1.3–1.5 dense parameters per parameter at 40M–160M, rising with data (§4.2)	Better: 8.0 against 4.2 steps solved with 4 loops (§6.2)
Dense of equal parameters and nearly equal depth (18 against 20 layers)	Recovers about half of the advantage at 40M, not the part that grows with data (§4.3)	Not tested
Dense, equal training compute	Dense better in all 258 measured comparisons at the shared learning rate; with retuned rates, level once (§4.4)	Loop no worse than dense models of equal executed depth (2.4–4.4 times its parameters): 8.0 and 9.0 steps against 6.4 and 8.0 (§6.2)
Written chain of thought	Not tested	Chain of thought better within the range: perfect, at under a third of the 8-loop model's inference compute, supervised on every intermediate value (§6.2)
Deployment, equal loss	4–19% less weight memory; 2.2–2.5 times the KV cache and decoding time (§4.5)	Not measured
Claim two: more loops at inference solve harder problems		
Its own training loop count	Never better (§5)	Longer chains: the 8-loop model goes from 0.04 to 0.83 at 12 steps with 16 loops; in most regimes its confidence tells it when to stop (§6.3, §6.6)
Other extra computation	Not tested	Dense models of equal depth, a chain of thought (whose prompt grows with the chain) and pause tokens do not extend beyond the range (§6.3)

2 Related work

Claim one: what a loop is worth. Sharing parameters across layers trades compute for parameters (Dehghani et al., 2019; Lan et al., 2019; Bae et al., 2025a); transformers that re-use parameters "perform slightly better when comparing models with equal parameter count, but slightly worse when accounting for reuse and comparing per FLOP" (Kaplan et al., 2020, App. D.2, Fig. 17). The law we test, $N_{\text{eff}} = N_{\text{once}} + r^{\varphi} N_{\text{rec}}$, and our layout come from Schwethelm et al. (2026), who estimate $\varphi = 0.46$ on FineWeb-Edu and find that backpropagating through only the last $\lceil r/2 \rceil$ loops, as Prairie et al. (2026) do, lowers it from 0.45 (their baseline refitted on the same budgets) to 0.38 in a comparison that spends the saved FLOPs on more tokens; our H1 separates the two accountings. Parcae finds that looping lowers perplexity at matched parameters and tokens, and that at a fixed FLOP budget looping should grow with data; dense looped models scale worse than non-looped ones at equal FLOPs (Lee et al., 2026); with repeated data the best loop count grows with compute (Chen et al., 2026b); a bounded form extrapolates better than r^{φ} (Chen et al., 2026a); looping the middle half of the layers beat looping all of them by less than 0.01 nats in single-seed ablations (Wang et al., 2026b); a dense model of the same depth beats the looped one in a controlled appendix of Ouro (Zhu et al., 2025), plain recursion is worse than a vanilla model at matched compute (Bae et al., 2025b), and a 1.4B looped model beats a dense model of its own size on all 12 of its benchmarks but not a 3.9B dense model of matched inference compute (Marchenko et al., 2026). Loops may pay most when data is limited (Gülbahar et al., 2026). Without sharing, the KV cache and decoding time grow with the loop count (Wu et al., 2025; Monea et al., 2026); Ouro shares the cache during decoding only, full backpropagation through a fixed depth prevents sharing in Huang et al. (2026), and models pretrained so that only the first loop writes a cache lose no quality (Monea et al., 2026). None of these studies varies model size and tokens per parameter separately with several seeds, which is what our grid adds.

Claim two: loops as test-time compute. Recurrent networks solve harder instances by running more iterations, and can "overthink" (Schwarzschild et al., 2021; Bansal et al., 2022). Looped transformers gen-

eralise to longer inputs on algorithmic tasks with an input-dependent number of loops (Fan et al., 2025); recurrent layers with input injection, on top of position embeddings made for the task, help arithmetic generalise further to longer numbers (McLeish et al., 2024); recurrent-depth transformers extrapolate to deeper chains of composed facts with more loops, while vanilla transformers of equal parameters or FLOPs do not (Kohli et al., 2026); and the positions a loop resolves per iteration scale with the training length divided by the training loop count (Zhang et al., 2026). Shallow transformers can learn shortcuts for associative serial computations (Liu et al., 2022). Looped and equilibrium models settle into fixed points or orbits (Geiping et al., 2025; Logan, 2026; Bai et al., 2019), those whose fixed point does not depend on the starting state use extra test-time computation better (Anil et al., 2022), and looping gains concentrate on reasoning tasks (Saunshi et al., 2025); we read the state after every loop as the logit lens (nostalgebraist, 2020) and the tuned lens (Belrose et al., 2023) read layers. A chain of thought (Wei et al., 2022) adds serial computation to a fixed-depth transformer (Merrill & Sabharwal, 2024); we learn it from scratch, as scratchpad training does for pre-trained models (Nye et al., 2021), and compare it and pause tokens (Goyal et al., 2024), filler tokens that help on some algorithmic tasks (Pfau et al., 2024), with loops on the same problems. RELAY aligns loop iterations with chain-of-thought steps (Yu et al., 2025), continuous thoughts feed the hidden state back as input (Hao et al., 2025), and transformers that write tokens generalise poorly to longer inputs (Zhou et al., 2024; Cho et al., 2024). Adaptive computation time (Graves, 2016), PonderNet (Banino et al., 2021) and Mixture-of-Recursions (Bae et al., 2025b) learn when to halt during training, and Fan et al. (2025) take the loop count at which the model is most confident; we compare answer-free stopping rules applied after training, fixed in advance. Our chain-task results replicate, with five seeds, that looped models solve longer chains with more loops (Kohli et al., 2026; Zhang et al., 2026); we add probes of every intermediate value, the effect of the learning rate, the stopping rules, and comparisons on the same problems with dense models of equal compute and with a written chain of thought.

3 Setup

3.1 Models

All language models (Figure 2a) are decoder-only transformers with eight unique layers of width w: pre-norm RMSNorm, SwiGLU feed-forward blocks (hidden size $\frac{8}{3}w$), rotary position embeddings, attention heads of dimension 64, no biases, untied input and output embeddings, a 16k BPE vocabulary trained on the training data, and a context of 1024 tokens. A *dense* model runs its eight layers once. A *middle-block* looped model runs two layers once (the prelude), then the middle four layers r times (the core), then two layers once (the coda), so it executes $2 + 4r + 2$ layers; a *whole-stack* looped model runs all eight layers r times. Each loop starts from a linear map of the previous state concatenated with the prelude output (input injection, $2w^2$ parameters, as in Huginn (Geiping et al., 2025)); the first state is zero. Every residual branch is scaled by $(8/12)^{-1/2}$ (eight layers against a 12-layer reference) and by a further $1/r$ inside the core, meant to keep the best learning rate independent of r (Wang et al., 2026a) (Appendix B). Backpropagation runs through all loops or, for *truncated* cells, through the last $r/2$ loops only. The five sizes, named by their approximate non-embedding parameters (10M to 160M), have widths 320 to 1280 and N = 15.4M, 26.8M, 50.1M, 92.7M and 179.6M, where N also counts the output head (Porian et al., 2024), which is large at small widths with a 16k vocabulary (Table 6 in Appendix B).

3.2 Data, training and the grid

Every run reads a prefix of one fixed-order stream, files 000–012 of FineWeb-Edu sample-10BT (Penedo et al., 2024) (10.4B tokens with our tokenizer; the longest run uses 9.9B), so no run sees a document twice. The main validation set is 50M held-out FineWeb-Edu tokens from file 013; three further sets of 20M tokens come from SlimPajama (Soboleva et al., 2023), FineMath (Ben Allal et al., 2025) and GitHub code (Appendix B). We use AdamW ($\beta = (0.9, 0.95)$, weight decay 0.1, gradient clipping 1.0), a learning rate per width from a sweep, which the looped models share (Appendix B), and a warmup–stable–decay schedule with cooldown branches (Hägele et al., 2024): one constant-rate trunk per run, from which cooldowns of 20% of the budget, decaying linearly to zero, branch off at several budgets. A budget is given in training tokens per parameter, written $10N$ for ten tokens per parameter. Looped runs have budgets of $10N$, $20N$ and $40N$; dense runs $5N$

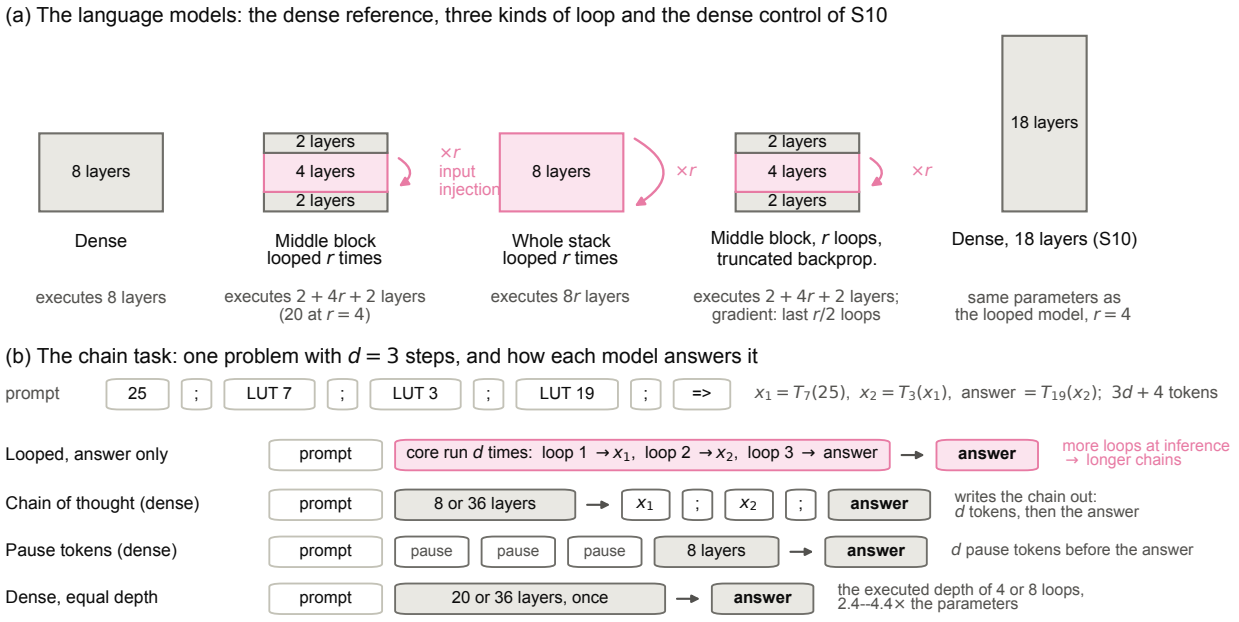


Figure 2: The models and the chain task. **(a)** Every language model has eight unique layers; a middle-block looped model runs its middle four layers r times, each loop starting from a linear map of the previous state concatenated with the prelude output (input injection), a whole-stack model runs all eight; with truncated backpropagation only the last $r/2$ loops receive gradients. S10 trains dense models with 18 layers and the looped model's number of parameters. **(b)** A chain-task problem with $d = 3$ lookups in fixed random tables T_k. The looped model answers in one forward pass, computing the chain inside the loop and solving longer chains when run for more loops; a dense chain-of-thought model writes the d intermediate values before the answer; a dense model with d pause tokens, and a dense model of the looped model's executed depth, answer in one pass.

to $80N$ ($40N$ at 160M). Each looped size has ten cells: middle block with $r = 2$, $r = 4$ and $r = 8$, each with full backpropagation and, for $r = 4$ and 8, truncated; and the same five for the whole stack. With one dense model per size, three seeds at 10M–40M and one at 80M, and a 160M dense reference, the pre-registered grid has 111 runs. Seeds change the initialisation, not the data order.

Extensions, reported as exploratory, add a 160M middle block looped 4 times and a second 160M dense seed, continue 10M models to $160N$ and 20M models to $80N$ (Appendix B lists the cells and seeds), and train 20M models with a random loop count; ten supplementary experiments, S1–S10, ran under plans frozen before their runs (Table 7 in Appendix B). Everything ran on four RTX 4090 GPUs, about 19 days of wall-clock time in all.

3.3 Measurement

Losses are in nats per token on the main validation set. Between initialisation seeds the loss of the 20M dense model varies by 0.014–0.024 nats at $10N$–$40N$ (0.003–0.07 across the looped cells at 20M); a difference of 0.01 nats between runs that share a seed is small but resolvable, one between unpaired runs is not.

The pre-registered law. The effective size of a looped model is $N_{\text{eff}} = N_{\text{once}} + r^{\varphi} N_{\text{rec}}$ (Schwethelm et al., 2026), where N_{once} counts the parameters run once and N_{rec} those in the loop; unlike Schwethelm et al. (2026), who count non-embedding parameters only, we include the output head in N_{once}, as in N. A dense "ruler" $L(N, D) = E + AN^{-\alpha} + BD^{-\beta}$ is fitted to the dense runs (Hoffmann et al., 2022; Besiroglu et al., 2024) (Huber loss in log space, block-bootstrap intervals; Appendix C); with the ruler fixed, φ is fitted per cell. A joint fit of the ruler and all φ is the pre-registered secondary analysis.

Table 2: Pre-registered results: φ per cell on the main validation set (equal-token accounting, N including the output head; primary analysis with 95% block-bootstrap intervals, and the secondary joint fit), and below, the verdicts on the four hypotheses under the frozen decision rules (in full in Table 5).

Cell	φ, primary [95% interval]	φ, joint fit
Middle block, full backpropagation	0.13 [0.02, 0.27]	0.41
Middle block, truncated	0.00 [0.00, 0.17]	0.29
Whole stack, full backpropagation	0.00 [0.00, 0.16]	0.00
Whole stack, truncated	0.17 [0.10, 0.20]	0.29

Hypothesis	Verdict
H1: truncation lowers φ	Not supported
H2: a middle block has a higher φ than the whole stack	Depends on the measure
H3: φ saturates in r	Not supported
H4: the best r is small under training compute, larger under deployment	First half trivially; second not supported

Without a fitted law. Three quantities need no ruler. The *gap* is the looped minus the dense loss at equal parameters and tokens, same seed. The *dense size of equal loss*, $N^\star$, is the size of a dense model that would reach the looped model's loss on the same number of training tokens: we read each measured dense size's loss at that token count and interpolate $\log N$ linearly in the loss. This gives the multiplier $M = N^\star/N$ ($M > 1$: the looped model beats a dense model of its own size) and a local exponent φ_{loc} defined by $N^\star = N_{\text{once}} + r^{\varphi_{\text{loc}}} N_{\text{rec}}$. The *equal-compute gap* converts a looped run's training FLOPs (counted over executed layers, including attention and the output head) into the tokens a dense model of the same size could have seen for them, and reads the dense loss there off the measured dense curve; where that point lies beyond the measured curve, two extrapolations bound it (Section 4.4).

3.4 Hypotheses and labels

The pre-registration, frozen on 2026-09-23 before any training run, states four hypotheses with decision rules (Appendix A): **H1** truncated backpropagation lowers φ, under both an equal-token and an equal-FLOP accounting; **H2** a looped middle block has a higher φ than a looped whole stack; **H3** φ saturates in r, so that a geometric form $N_{\text{eff}} = N_{\text{once}} + N_{\text{rec}}(1-\rho^r)/(1-\rho)$ extrapolates better than r^φ to a held-out size; and **H4** the best r is small under a training-compute budget and larger under a deployment-compute budget. Throughout, results are labelled *pre-registered* (hypothesis and rule frozen before training), *pre-specified* (written in a plan frozen before the runs it describes: the synthetic design on 2026-10-09 and the three supplementary plans on 2026-10-13, 2026-10-15 and 2026-10-27) or *exploratory* (chosen after seeing data). Every departure from a frozen document is dated in the deviation log.

4 What a loop is worth on text

4.1 The pre-registered law cannot say what a loop is worth

The pre-registration asked what a loop is worth through one number per cell, the exponent φ of $N_{\text{eff}} = N_{\text{once}} + r^\varphi N_{\text{rec}}$, read off a dense ruler fitted to the 54 dense results of the grid. That ruler puts its irreducible loss at the lower bound of the fit ($E = 0.050$) and its parameter exponent at $\alpha = 0.054$, far below the usual value of about 0.3; five sizes cannot pin down E, A and α separately. φ inherits the weakness: it is 0–0.17 per cell with the primary fit and 0–0.41 with the joint fit over all 352 results, which gives $\alpha = 0.267$ (Table 2). None of the four hypotheses held as stated *(pre-registered)*. Truncated backpropagation, predicted to lower φ (H1), pointed that way in the middle block, with overlapping intervals, and the other way in the whole stack, where it also lowered the loss at every size with four loops; it brought most of the loss spikes as well (Appendix B). The full verdicts are in Appendix A. What survives is a trend: with the ruler fixed, the middle block looped 4 times has $\varphi = -0.32$, 0.19, 0.39, 0.52 and 0.65 from 10M to 160M *(exploratory)*, so a single φ per cell averages over very different regimes. We therefore measure the value of a loop without a fitted law.

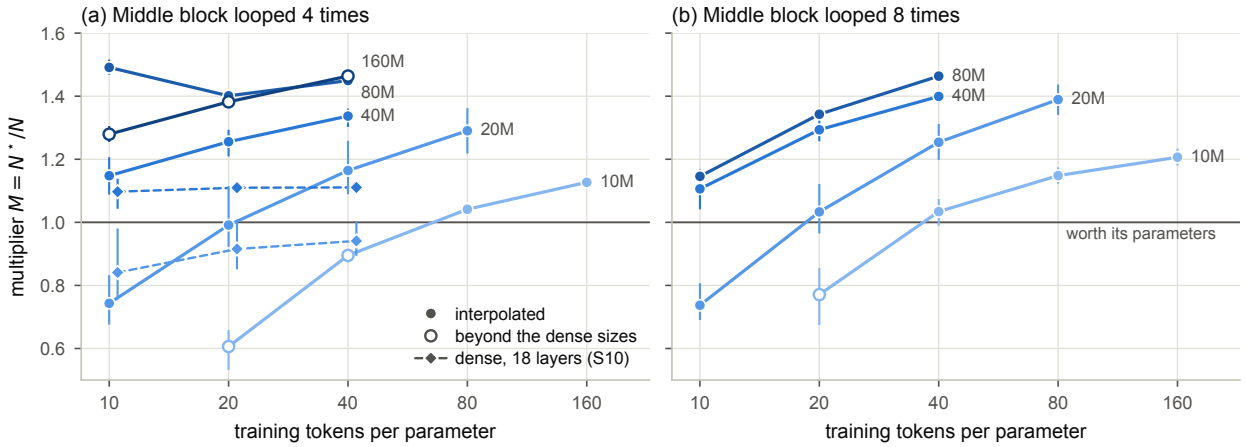


Figure 3: What each parameter of a model with a looped middle block is worth: the multiplier $M = N^\star/N$, where $N^\star$ is the size of the dense model that reaches the same loss on the same number of training tokens (interpolated between the measured dense sizes, including the dense models continued in S4; open markers: beyond the largest or smallest by at most a factor of two). $M = 1$: as good as a dense model of its own size. Diamonds: dense models with 18 layers and the looped model's number of parameters (Section 4.3). Bars: 95% intervals from resampling seeds (80M has one seed).

4.2 Measured directly, a loop's value rises with model size and data

The dense runs themselves can serve as the ruler. For each looped model we read the loss of every measured dense size at the looped model's number of training tokens and interpolate the dense size $N^\star$ with the same loss; $M = N^\star/N$ is what each parameter of the looped model is worth in dense parameters (Figure 3, *exploratory*). For the middle block looped 4 times, M rises from 0.6–1.0 at 10–20M with 10–20 tokens per parameter to 1.26–1.46 at 40M–160M with 20–40 (the 160M values lie beyond the largest dense model, by less than a factor of two); at 10M with 10 tokens per parameter it cannot be read off, because the looped model is worse than both dense models measured at that number of tokens. A least-squares plane in $\log_2 M$ gives a factor of 1.21 per doubling of model size and 1.14 per doubling of tokens per parameter (Appendix C). Eight loops add up to 0.16 over four at 10M–20M and nothing consistent from 40M on. At 40M–160M with 20–40 tokens per parameter, counting N without the output head as Schwethelm et al. (2026) do, the local exponent is $\varphi_{\text{loc}} = 0.32$–$0.50$, close to their $\varphi = 0.46$.

The shared learning rate does not create the trend. Re-tuning the dense model and the middle block looped 4 times at 10M–40M with one seed (two at 20M; supplementary experiment S1, *pre-specified*; Appendix D) moved the gap between them by at most 0.005 nats at 20M–40M and about 0.02 at 10M, where the looped model preferred lower rates; with each at its best rate, the size trend at $40N$ is somewhat smaller but holds by the criterion of the plan, and the data trend at 10M is larger ($+0.082$ at $10N$ to -0.059 at $160N$). Published equal-parameter results, restated in the same unit, mostly agree in comparable settings (1.23–1.46 at 134M–770M) and go higher with larger models or far more tokens per parameter (Appendix C); they barely sample models below 40M trained on 20 or fewer tokens per parameter, where we find the loop worth at most its own parameters. That the value rises with tokens per parameter is a dependence that effective-parameter laws whose loop gain does not depend on the training tokens (Schwethelm et al., 2026; Chen et al., 2026a) do not capture.

Every dense model of this ruler has eight layers, while a middle block looped 4 times executes 20, and at a fixed size deeper and thinner models do better below a billion parameters (Liu et al., 2024), though the shape matters little over a wide range of sizes (Kaplan et al., 2020). Is M what sharing buys, or what depth buys at a fixed size?

4.3 At 40M, a deeper dense model of the same size recovers about half of it

Supplementary experiment S10 *(pre-specified)* trained dense models with 18 layers and the looped model's number of parameters (18×320 with $N = 28.1$M at 20M, 18×448 with $N = 51.2$M at 40M; executed depth 18 against the looped model's 20) with the recipe, data, budgets and three seeds of the grid and the learning rate its rule gives them (the frozen rate of their width, with the hidden layers scaled by $(18/12)^{-1/2}$; below, the rule's rate), and read their multiplier M_{deep} off the same 8-layer dense runs. Its plan fixed the reading in advance: $R = \log M_{\text{deep}} / \log M_{\text{loop}}$ at 40M, averaged over the three budgets, is the share of the looped model's advantage that a dense model of the same size and depth recovers.

At 40M the deep dense model is worth 1.10, 1.11 and 1.11 dense parameters per parameter at $10N$, $20N$ and $40N$, against 1.15, 1.26 and 1.34 for the looped model (Figure 3a, diamonds). R is 0.50 (95% interval 0.41–0.62, resampling seeds), in the band that the plan reads as "partly depth, partly sharing", as it predicted. At 40M, what depth buys does not grow with data; the looped model's lead over the deep dense model does, from 0.003 nats at $10N$ to 0.015 and 0.029 at $20N$ and $40N$, with the same sign in all three seeds at the two longer budgets. At 20M the deep model, narrow at width 320, is worse than the 8-layer one of the same size ($M_{\text{deep}} = 0.84$–0.94), so there depth explains none of the looped model's advantage at $40N$ ($M_{\text{loop}} = 1.16$; the plan reports 20M descriptively).

The learning rate qualifies the shortest budget. At 40M the deep model was best at 1.41 times the rule's rate, the highest rate tried (seed 42), at all three budgets, by 0.014, 0.006 and 0.001 nats. With each model at its best tested rate, as the plan asks in that case, the deep model is ahead of the looped one at $10N$ by 0.010 nats ($M = 1.37$ against 1.18), and at $20N$ and $40N$ R is 0.56 and 0.34 (0.45 and 0.37 for the same seed at the rule's rate); since its best rate lies at the edge of the range tried at all three budgets, these readings may understate it. Averaged over the three budgets, R at the best rates is 0.94, in the band the plan reads as "most of the advantage is a depth effect"; the average is carried by $10N$, where $\log M_{\text{loop}}$ is smallest, and the plan keeps the primary reading at the rule's rate. The deep model also costs about half the training compute of the looped model (406M against 835M FLOPs per token at 40M). So at 40M about half of what the loop buys at equal parameters is depth, and at the shortest budget, once the deep model's learning rate is tuned, all of it (one seed, by 0.010 nats); at 20M none of it is; and the part that grows with data is not recovered by depth (Appendix E). What the loop buys at equal parameters, it pays for in compute.

4.4 At equal training compute, the dense model wins all but one measured comparison

At equal parameters, a middle block looped 2, 4 or 8 times costs 1.4–1.5, 2.2–2.4 or 3.7–4.1 times the training FLOPs per token of the dense model. For an equal-compute comparison we give the dense model the tokens it could have seen with the looped run's compute and read its loss off the measured dense curve *(exploratory)*. Of the 320 looped results of the grid and its extensions, 221 could be compared this way with the original dense runs, and the dense model won all of them, by at least 0.016 nats; the other 99 needed the dense loss beyond its measured curve, which two extrapolations bracket (Appendix D), and with the one that flatters looping, looping came out ahead in 6 of them, by at most 0.021 nats, all at the largest data per parameter.

Continuing the 10M dense models to $640N$ and the 20M ones to $320N$ (supplementary experiment S4, *pre-specified*) turned 37 of the extrapolated comparisons into measured ones (Figure 4). The dense model wins all 258 measured comparisons, which share dense curves, trunks and seeds and are not independent, and the measured gaps fall about halfway between the two extrapolations. The margin shrinks with data per parameter: for the 10M middle block looped 4 times it is +0.119 at $40N$, +0.062 at $80N$ and +0.016 at $160N$ (Table 11 in Appendix D). With each model at its best tested learning rate (S1), the dense model won 10 of the 11 comparisons (one of them, 40M at $40N$, on an extrapolated dense loss); the eleventh, at 10M and $160N$ with the looped model at 0.35 times the dense rate, the lowest rate tried, where its loss still fell as the rate was lowered, is level on average (−0.011 and +0.005 in two seeds): the only measured equal-compute comparison in this study that the dense model does not win. The deep dense models of Section 4.3, at about half the looped model's compute per token, are ahead by 0.11–0.19 nats where their curves reach without extrapolation *(exploratory)*. The other 62 comparisons still need an extrapolated dense loss (Appendix D). Within the measured range, loops trade compute for parameters and data; they do not save compute.

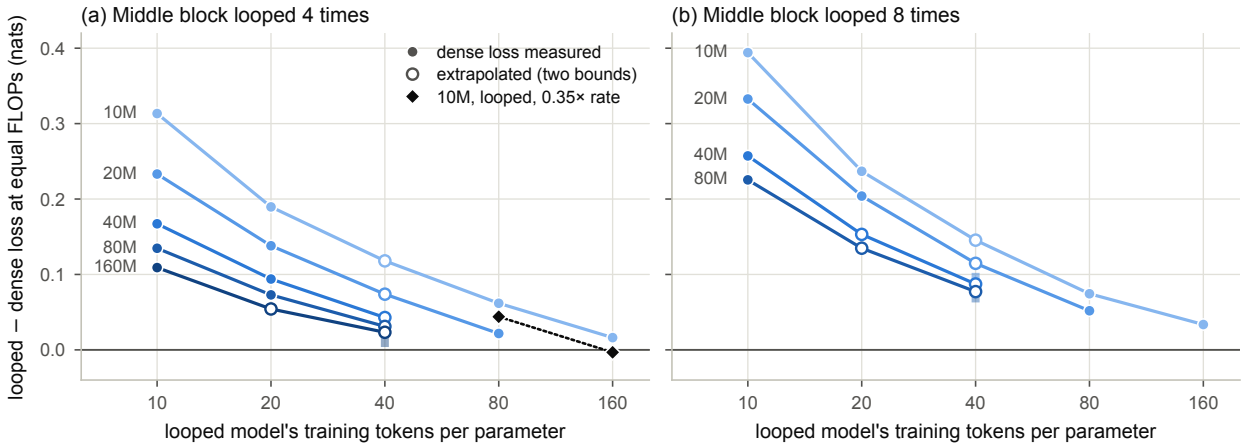


Figure 4: Equal training compute: looped minus dense loss when the dense model of the same size is trained on the tokens the looped run's compute would buy (positive: dense better). Filled markers: dense loss measured, including the dense models continued to $640N$ (10M) and $320N$ (20M) in S4. Open markers with bars: the dense loss is extrapolated; the bar spans the two bounds (power law to last segment). Diamonds: the 10M model looped 4 times trained at 0.35 times the dense rate, the lowest rate S1 tried (mean of two seeds; bars: seed range).

4.5 At deployment, the loop saves some weight memory and pays for every executed layer

We measured inference on one RTX 4090 in bf16 with KV-cached decoding and one cache per executed layer, the computation the models were trained with (supplementary experiment S3, *pre-specified*; Appendix F). At these sizes decoding time follows the executed layers: at equal parameters, a middle block looped 2, 4 or 8 times takes 1.5, 2.4–2.6 or 4.5–4.6 times the decoding time per token of the dense model, close to the ratios of executed layers. Against the dense model that reaches the same loss at $40N$, the middle block looped 4 times needs 4% less weight memory at 20M, rising to 19% at 160M (where the dense model of equal loss is extrapolated), but 2.20–2.41 times the KV cache per token, 2.37–2.47 times the decoding time and about 1.6 times the peak memory at batch 32. Sharing the KV cache across loops would remove most of the cache and memory cost, though not the decoding time, but applied after training to 20M models of one seed (supplementary experiment S6, *pre-specified*), it costs every model that gains from its loops 1.8 to 3.7 times that gain (0.09–0.17 nats against gains of 0.03–0.05), so that it would fall below the dense model of its size. Only the models trained with a random loop count, whose block contracts to a fixed point (Section 5) and which gain nothing over the dense model, share for free. Training towards convergence, or with a shared cache in the scheme of Monea et al. (2026), in which only the first loop writes a cache and which costs no quality there, may avoid this; we tested neither.

What the loop buys on text. At these sizes the loop trades compute for parameters and data. From about 40M on and with 20–40 tokens per parameter, each parameter of a model whose middle block is looped 4 times is worth 1.3–1.5 dense parameters, more with more data; at 40M a dense model of the same size and nearly the same depth gets about half of that advantage for half the compute, but not the part that grows with data. At equal training compute the dense model wins every measured comparison but one, and at deployment the loop pays for every executed layer. The second claim, that more loops at inference buy test-time compute, is the subject of the next two sections.

5 On text, extra loops at inference never help

If the loop iterated towards an answer, the state after each loop would carry a better and better prediction, and running more loops at inference would help. We probed trained models loop by loop on trunk checkpoints at $32N$ *(exploratory)*, reading the state after each loop with the model's own output layers (a logit lens, here

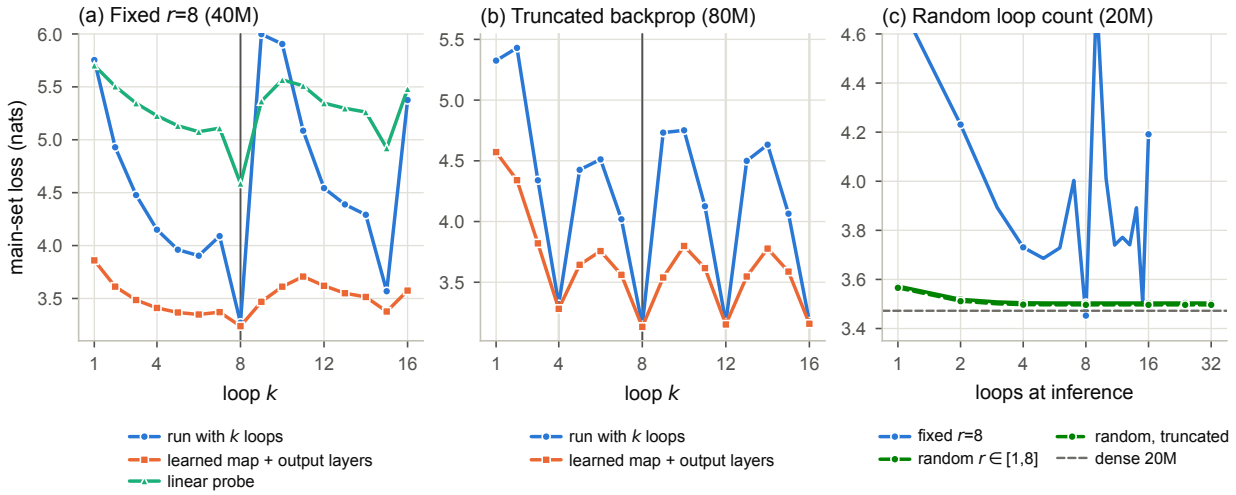


Figure 5: What the looped block computes on text (main-set loss of trunk checkpoints at $32N$; values above the top of a panel are clipped). **(a)** Fixed loop count: a 40M middle block trained with 8 loops (mean of three seeds), read after each loop k by running the model with k loops, by a learned map in front of the output layers, and by a learned linear head. The model works only at its training loop count. **(b)** Truncated backpropagation (80M whole stack, $r = 8$, gradients through the last 4 loops): the state returns to the same point every four loops. **(c)** A random loop count during training (20M middle block, r drawn from 1–8 at every step) contracts to a fixed point: flat from about four loops on, no better than the dense model, with or without truncation.

the same as running the model with that many loops), with a learned linear map in front of them (a tuned lens trained on the next token) and with a learned linear head (Appendix C). Figure 5 shows three training regimes.

A fixed loop count gives a fixed-length program. Trained with 8 loops, the model works only at 8 (Figure 5a). With a learned map, the state after loop 4 already reads out within 0.1–0.2 nats of the final loss, and loops 5–7 add little; the last loop is a large non-linear move that rewrites the state into what the output layers expect, and 37–47% of the tokens settle their prediction only there. Running fewer or more loops breaks the model. This held in all three seeds at 20M and 40M and at 80M (one seed).

Truncated backpropagation gives a cycle. With gradients only through the last four loops, the whole stack learns a periodic cycle (Figure 5b): the state returns to almost the same point every four loops, so the model works at 4, 8, 12 or 16 loops and gains nothing from more. The same minima at 4, 8 and 12 loops appear at 20M and in both probed 40M seeds.

A random loop count gives a fixed point. Drawing the loop count from 1–8 at every step, as Huginn does with a different distribution (Geiping et al., 2025), gives a block that contracts to a fixed point: the loss is flat from about four loops to 32 (Figure 5c; one seed at 20M, with full and with truncated backpropagation). The model is robust to the loop count, but at its nominal 8 loops it is 0.05 nats worse than fixed-count training and no better than the dense model; Parcae's models, trained with a sampled loop count at 140M–770M, keep an advantage, so this may be specific to small models or to our distribution (one seed at one size).

The learning rate shapes the iteration but never makes extra loops help. Lower learning rates give flatter curves over the inference loop count. In S1, on the middle block looped 4 times at 10M, 20M and 40M, the loss varies by 0.36–0.77 nats over 4–16 loops at 0.5 times the dense rate and by 3.4–4.6 nats at 1.41 times. At every rate, the better of 8 and 16 loops was worse than the training count of 4. On text, more loops than trained never helped.

The gain is a broad shift. Per token, the gain on web text is small and broad: about half of the tokens improve, and the gain is not concentrated on hard tokens beyond what two dense seeds differ by *(exploratory)*. The gains on math and code text are larger, but there the seed-to-seed spread is 10–100 times larger as well and the code set is more sensitive to the data order than to the initialisation (supplementary experiment S7, *pre-specified*; Appendix C), so we draw no conclusions from differences on those sets. Web text at these sizes thus gives the loop little reason to iterate. To test the second claim where it has one, we built a task that needs serial steps.

6 On a serial task, a loop buys extendable depth

We built a task that needs serial steps, froze its design, its conditions and its predictions before the main runs, and ran it with the same architecture. Pilot runs then led to three changes before any formal run, all for learnability: the operations became lookup tables instead of modular arithmetic, a curriculum was added, and training was lengthened; the learning-rate protocol was fixed after the pilot (Appendix H). We first ask what the loop buys at its training loop count (Section 6.2), then what extra loops buy (Section 6.3).

6.1 The task and the conditions

A problem lists a start value and a sequence of table lookups (Figure 2b), for example `25 ; LUT 7 ; LUT 3 ; LUT 19 ; ... =>`, where each `LUT k` is one of 32 fixed random functions on $\{0, \ldots, 96\}$. The model must output the final value in a single forward pass; the loss is taken at the answer position only. Two lookups compose into one of 1024 arbitrary functions, so a model that skips steps must memorise rapidly growing tables of compositions, although, as for any associative composition, a shallower solution exists in principle (Liu et al., 2022). A prompt with d steps has $3d+4$ tokens. Models are trained on 1–8 steps, sampled online with a curriculum that widens the range over the first half of training, for 32,000 steps of 512 problems, and are tested on 2000 fresh problems at every length from 1 to 24. Chains longer than 8 steps never occur in training; short problems do repeat, since there are only 3,104 one-step problems.

All models have width 256. The conditions are dense models of 8, 20 and 36 layers (6.5M, 16.1M and 29.0M parameters) and 8-layer looped models (6.6M parameters, a middle block of four layers looped r times) trained with $r = 2$, 4 or 8 loops, with a random loop count drawn from 1–8 at every step, or with as many loops as the chain has steps ("loops = steps", the regime of Fan et al. (2025)); a fourth regime, loops = steps with a few random extra loops (S9), follows in Section 6.6. The 20- and 36-layer dense models match the executed depth of the looped models with 4 and 8 loops. Supplementary experiment S5 *(pre-specified)* added dense models that write the intermediate values out as tokens before the answer, a chain of thought (Wei et al., 2022) trained from scratch on the written values as in scratchpad training (Nye et al., 2021), with 8 layers (the looped models' size) and 36 (their executed depth), and an 8-layer model with d pause tokens before the answer (Goyal et al., 2024). Each condition's learning rate was chosen by a sweep on in-range accuracy only; the looped rates came out 4–8 times lower than that of the 8-layer dense model, which has the same unique layers (Appendix G). Every condition has five seeds. The design fixed five predictions (P1–P5, Section 6.7); a planned pointer-chasing control was not learned by any condition tried on it and was dropped (Appendix G).

6.2 Within the training range, a loop buys depth, and a written chain of thought buys it more cheaply

Two caveats frame this comparison. A dense model of equal executed depth has the same width and is trained for the same steps on the same problems, so on this task equal executed depth means about the same training and inference compute, at 2.4–4.4 times the parameters. And the chain-of-thought models are trained with a loss on every intermediate value, the other models on the answer only, a difference the plan stated in advance; a looped model with intermediate supervision was outside the plan.

At equal parameters, more loops solve longer chains: 4.2, 6.4, 8.0 and 9.0 steps for the dense model and 2, 4 and 8 loops (Table 3, *pre-specified*); within the training range the 8-loop model is at 0.998 where the 8-layer dense model is at 0.62. At equal executed depth, and so at equal compute, the loop is at least as good: the 20-layer dense model solves 6.4 steps against 8.0 for 4 loops, though it did not fully learn the training

Table 3: The chain task, trained on 1–8 steps (seed means; chance is 0.01; accuracies rounded, so that only the chain-of-thought models are exactly perfect in range). Largest solvable length: mean (min–max) over five seeds of the longest chain length up to which every length is solved with accuracy ≥ 0.9. "Own": at the training loop count (d loops for loops = steps); "more": the best of the training and every larger inference loop count, including d loops, chosen per seed with the answers. Layer passes count the layers run over the tokens processed for a 12-step problem: a prompt of 40 tokens, plus 12 tokens written by the chain-of-thought models (with a KV cache) or 12 pause tokens.

			Within the range	Largest solvable length		Beyond the range	
Model	Params	Layer passes	Acc. 1–8 steps	Own loops	More loops	Acc. 12 steps own / more	Acc. 16 steps own / more
Equal parameters							
Dense, 8 layers	6.5M	320	0.621	4.2 (4–5)	—	0.06	0.05
Looped, $r = 2$	6.6M	480	0.906	6.4 (6–7)	8.6 (6–11)	0.05 / 0.64	0.04 / 0.33
Looped, $r = 4$	6.6M	800	1.000	8.0 (8–8)	11.8 (10–15)	0.03 / 0.83	0.03 / 0.28
Looped, $r = 8$	6.6M	1440	0.998	9.0 (9–9)	11.2 (11–12)	0.04 / 0.86	0.03 / 0.37
Looped, random loop count	6.6M	1440	0.999	11.4 (10–13)	12.2 (10–15)	0.77 / 0.83	0.08 / 0.49
Looped, loops = steps	6.6M	2080	1.000	12.8 (11–16)	—	0.96	0.37
Loops = steps + 0–4 extra (S9)	6.6M	2080	0.999	11.6 (11–12)	—	0.90	0.31
Equal executed depth, which here is equal compute							
Dense, 20 layers (as $r = 4$)	16.1M	800	0.910	6.4 (6–7)	—	0.04	0.03
Dense, 36 layers (as $r = 8$)	29.0M	1440	0.999	8.0 (8–8)	—	0.02	0.02
Dense models with written-out reasoning or pause tokens (S5)							
Chain of thought, 8 layers	6.5M	416	1.000	8.0 (8–8)	—	0.01	0.01
Chain of thought, 36 layers	29.0M	1872	1.000	8.0 (8–8)	—	0.02	0.01
d pause tokens, 8 layers	6.5M	416	0.566	4.0 (4–4)	—	0.01	0.01

range (0.91), and the 36-layer one 8.0 against 9.0 for 8 loops, level within the range. Unlike on text, where the dense model of equal compute has the looped model's size and more data, here it has 2.4–4.4 times the parameters and the same data, and it does not beat the loop; we do not read these two comparisons as a saving of compute, since one dense model did not fully learn the range and the other is level within it. A written chain of thought is cheaper: it is perfect within the training range, 1.000 in every seed and at every learning rate tried, where the 8-layer model that answers directly solves 4 steps, and it needs under a third of the inference compute of the 8-loop model at any length (416 against 1440 layer passes at 12 steps). Pause tokens add nothing: the pause model solves the same 4 steps as the model without them. Within the training range, the loop buys serial depth at the price of its executed layers, and written tokens buy it more cheaply.

6.3 Beyond the training range, only the loop extends

All answer-only dense models fail right after the training range: at 12 steps they score 0.02–0.06, and so do the fixed-loop models at their training loop count, 0.03–0.05 (Figure 6a). Run with more loops, the looped models solve longer chains (Figure 6b). The model trained with 8 loops solves 9 steps at 8 loops and about 11 with 16 or 32; at 12 steps its accuracy rises from 0.04 to 0.86 with the best loop count of 12–32 chosen per seed (0.83 at 16 loops, on average). The gain is real but bounded: at 16 steps the best is 0.37, with a wide spread across seeds. The model trained with a random loop count gains as well (0.08 to 0.49 at 16 steps), against the frozen prediction that it would not. The model trained with loops = steps, run with as many loops as the chain has steps, solves the longest chains at accuracy 0.9 (12.8 steps) and beats 8 loops by 0.34–0.95 at every length from 10 to 16 (counting from 10 steps, a reading fixed after the results; at 9 steps the gain is 0.03), partly because the problem states its number of steps.

The written chain of thought does not extend at all: 0.014 at 12 steps with 8 layers and 0.017 with 36, near the chance level of 0.010, on prompts on which the looped models reach 0.83–0.96 (Figure 6c). It breaks at the start: as soon as the prompt holds more than 8 tables, the first wrong token is one of the first three written ones in 97–98% of the problems for the 8-layer model *(exploratory)*. The failure is one of generalising

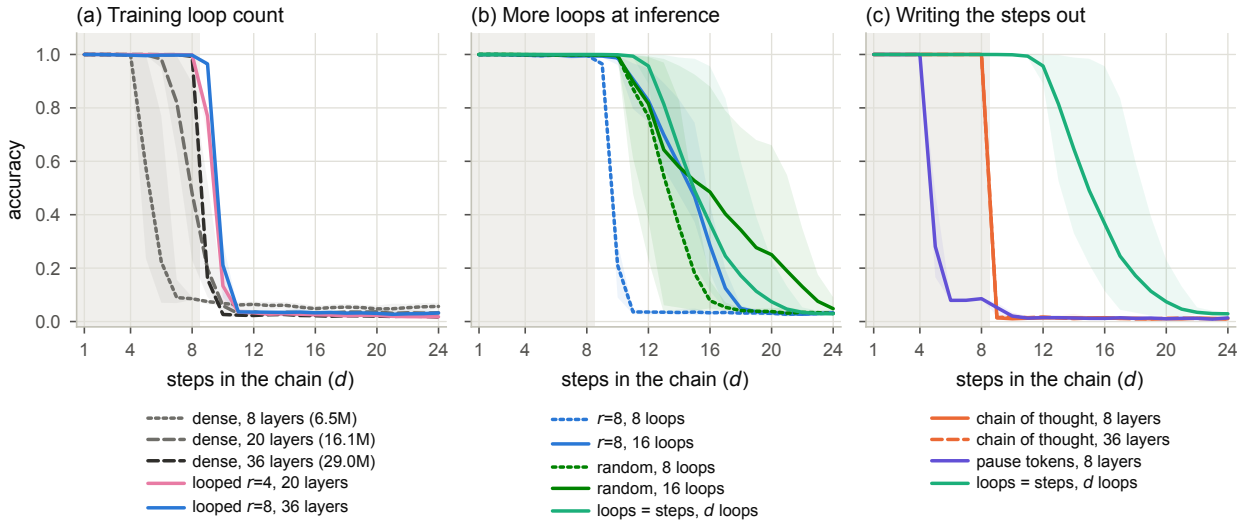


Figure 6: The chain task, trained on 1–8 steps (shaded); accuracy against chain length, mean over five seeds (band: range). **(a)** At the training loop count, looped models solve at least as far as dense models of the same executed depth, and none gets more than one step past the training range. **(b)** With more loops at inference, the looped models solve longer chains; the model trained with as many loops as steps, run with d loops, solves the longest chains at accuracy 0.9. **(c)** Dense models that write the intermediate values out are perfect within the training range and near chance beyond it; pause tokens add nothing.

to longer inputs, a known weakness of transformers (Zhou et al., 2024; Cho et al., 2024), not an accumulation of errors along the chain. Because the prompt of this task grows with the number of steps, the comparison cannot say whether a written chain would extend on a task whose input length does not grow, and remedies for length generalisation were outside the plan. Pause tokens do not extend either.

Together with Section 6.2, this is what the loop buys on this task: not the cheapest computation within the training range, but depth that extends beyond it. What the answer-only dense models lack is not depth but the option to run longer; the chain of thought has that option and fails on the longer prompt.

6.4 The loop computes the chain step by step

For each intermediate value x_j we trained a linear probe on the state after every loop, at the input positions of step j, on problems with 8 steps (planned in the frozen design; the summaries are *exploratory*; Figure 7). In the model trained with 8 loops, x_j first becomes decodable (probe accuracy ≥ 0.5) at about loop j, 1.0–1.1 loops per step in all five seeds, and stays decodable afterwards (lowest later accuracy, averaged over the values, 0.96). The loops = steps model also computes one step per loop and keeps its values for its d loops (0.92), but loses them when run longer: it was never trained to keep looping after the last step, which is why it needs exactly d loops. Models with fewer loops than steps pack about 1.6 steps into each loop with 4 loops and 2.4 with 2, the direction predicted by the budget law of Zhang et al. (2026). The dense models compute late (from about layer 21 of 36), and in 5 of their 10 seeds at 20 and 36 layers the probes read no intermediate value after the first. This fits the extension: a loop that applies the same step each time can be run for more steps, a stack that computes the chain in its last layers cannot. Probes show where a value can be read out, not that the model uses it.

6.5 The learning rate decides whether the loop learns the task within the budget

At 10^{-3}, a good rate for the dense models, the 8-loop model of the learning-rate sweep worked only at exactly 8 loops and had no step-by-step frontier (no intermediate value became decodable one loop per step), like the language models of Section 5, but it had also learned the training range much worse (0.54 against 0.999

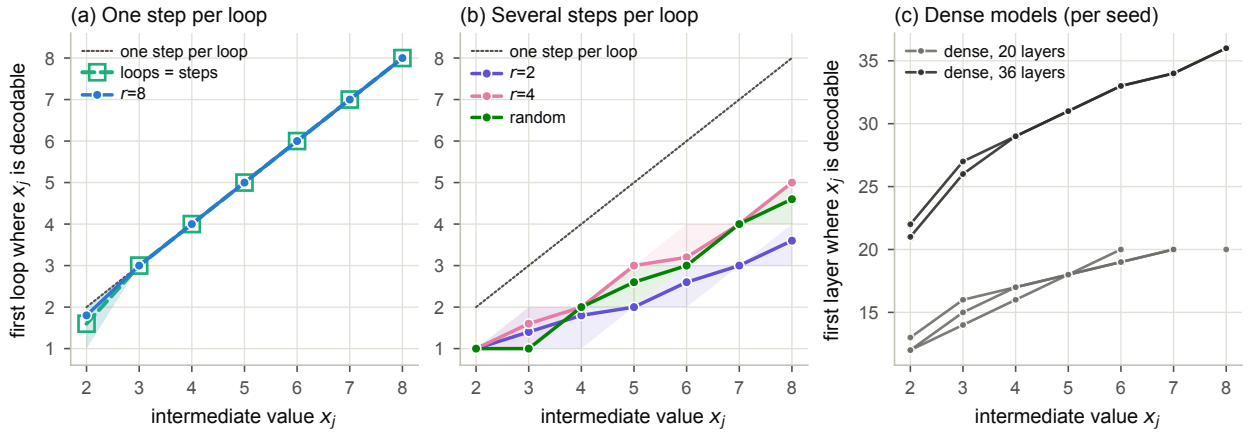


Figure 7: Where the intermediate values appear (probes on 8-step problems; a value counts as decodable at probe accuracy ≥ 0.5). **(a)** With a loop for every step, the chain advances one step per loop. **(b)** With fewer loops than steps, each loop advances several steps (mean over seeds; band: range). **(c)** Dense models compute the values late in their depth (one line per seed; a seed in which only x_1, not plotted, and the answer are decodable appears as a single point at the last layer; the 20-layer seed with no decodable intermediate value is not shown).

Table 4: Stopping without the answer: accuracy on 12-step chains (seed means, reporting set; mean loops used, where given, in parentheses, and for every rule in Table 15). The best fixed loop count is chosen with the answers; it is a reference, not a rule, and an adaptive rule can exceed it ($r = 2$).

Model	Own loop count	Best fixed count	State change $< 2\%$	State change $< 0.5\%$	Confidence ≥ 0.99
Looped, $r = 2$	0.05	0.62	0.45	0.45 (32)	0.70 (8.3)
Looped, $r = 4$	0.04	0.83	0.65	0.82 (27)	0.81 (9.2)
Looped, $r = 8$	0.04	0.86	0.71	0.84 (28)	0.84 (17)
Random loop count	0.77	0.83	0.73	0.83 (22)	0.83 (14)
Loops = steps (d loops)	0.96	0.96	0.03	0.03 (29)	0.76 (14), 6–7% lost in range
Loops = steps + 0–4 extra (S9)	0.90	0.94	0.52	0.94 (24)	0.93 (14), 0.998 kept in range

at 2.5×10^{-4}). Trained for 32,000 steps with three seeds (supplementary experiment S2, *pre-specified*), the four looped conditions that its plan had identified as extrapolating did not learn the training range in 10 of 12 runs at 10^{-3} and learned it in 8 of 12 at 5×10^{-4}, and those mostly extended about as far as the formal runs (Appendix G). A high learning rate mainly keeps the looped model from learning the task in the given steps; when it learns the task, it mostly learns the same extendable algorithm.

6.6 The model's own confidence tells it when to stop

Extra loops help only if the model can tell how many to run. The frozen design specified one answer-free rule, for the loops = steps model: stop when the state changes by less than a threshold between loops, implemented at the freeze as a relative change below 2% at the answer position, with at most 32 loops. Supplementary experiment S8 *(pre-specified*; planned after this rule's results were known, frozen before the comparison) compared it on the same 25 checkpoints with rules that stop when the prediction has not changed for m loops or when the model's top probability reaches a threshold (the early-stopping variant of the maximum-confidence rule of Fan et al., 2025), and with votes over loop counts, all fixed in the plan, selected on one problem set and reported on another (Table 4).

The state rule recovers about 80% of the gain of the best fixed loop count at 12 steps for the models trained with 4 or 8 loops, without hurting the training range, but it cannot stop the loops = steps model, whose state keeps changing after d loops (0.02–0.08 at every length). The model's own confidence does better: at a threshold of 0.99 it recovers 97–98% of that gain for the models trained with 4 or 8 loops or a random count, using fewer loops than their training count on in-range problems, and brings the loops = steps model to 0.76 at a cost of 6–7% of the in-range problems. Among the rules that stop, it is the best in every condition of S8 except $r = 4$, where a state rule with a lower threshold (0.5%) ties it (0.816 against 0.815) with far more loops (27 against 9). Stopping when the prediction stops changing fires on stable wrong answers, and votes over loop counts run all 32 loops and never beat the best fixed count; the plan's selection criterion picked a vote for three of the five conditions and, without votes, the confidence rule at 0.99 only for $r = 8$ and the random loop count (Appendix G).

The models that extend furthest and the models that stop well thus came from different training regimes. Trained with 0–4 random extra loops after the last step (supplementary experiment S9, *pre-specified*), the loops = steps model keeps its values when run longer (lowest probe accuracy over 16 loops 0.04 → 0.91), extends about one step less (11.6 against 12.8), and stops by confidence at 0.93 on 12-step problems while keeping 0.998 of the training range; the 0.5% state rule is slightly better (0.94, in all five seeds) with 24 loops against 14. By the frozen state rule its stop is seed-dependent (0.52: it works in two seeds, fails in two and half-works in the fifth), an outcome none of the three interpretations written into the plan covered. Training it costs about 1.4 times the compute per step.

6.7 Against the pre-specified predictions

Of the five predictions of the frozen design (Table 14 in Appendix G), P5 (much larger gaps than on text) holds; P1 (more loops, more steps) holds in direction, but the 8-loop model did not reach the predicted 16 steps; P2 (dense models of equal depth at least as good) is not supported as stated, with gaps of 1–1.6 steps in favour of the loop; P3 (inference-time behaviour) is half right in its first part, fails in its second in the direction that favours looping (the random-loop model gained) and holds in its third (the loops = steps model extends); and P4 (only the loops = steps model computes along the diagonal) is half right, since the 8-loop model does as well, though by the design's literal reading P4 would hold. The readings of P3's third part and of P4 were fixed after the results. Of the supplementary predictions on this task, S5 held one of three, S8 two of five and S9 three of four.

What the loop buys on a serial task. On a task that needs serial steps, the same architecture computes the chain step by step inside the loop, one step per loop when it has a loop for every step. Within the training range this buys depth, at no less compute than a dense model of the same executed depth and at more than a written chain of thought; beyond it, the loop is the only form of extra computation here that extends. It learns to do so when its learning rate is low enough for the task to be learned in the budget, and, for most training regimes, its own confidence tells it when to stop.

7 Discussion: when is recurrent depth test-time compute?

The same architecture behaves as a parameter-efficient model on text and as an extendable serial computer on the chain task. We offer a reading of the difference. It is an interpretation of two settings, not a tested law: in this reading, recurrent depth works as test-time compute when three conditions hold.

Headroom. The model's errors must come from too few serial steps rather than from missing knowledge. On the chain task the dense models know every table, and within the training range their errors come from too few serial steps; one more loop is one more step. On web text at these sizes we found no sign of such errors: the per-token gain is a diffuse shift, and extra loops never helped, at any learning rate. This condition is the hardest to measure independently, since here it is inferred from where extra loops help; the larger gains on math and code text hint at more headroom there, but at this size those sets are dominated by the initialisation and the data order.

Extendable iteration. The loop must be able to run longer without destroying its state, and which kind of iteration it learns is decided by its training more than by the architecture: on text the same block becomes a fixed-length program, a periodic cycle or a fixed point, depending on the loop schedule; on the chain task, models trained with as many loops as steps lose their values when run longer, and a few random extra loops in training fix that. A fixed point is extendable but, on text, computes nothing more; a fixed-length program computes but cannot be extended.

A stopping signal. Extra loops are only useful if the model can tell how many to run. On the chain task the model's own confidence is such a signal for most regimes, including, once trained with a few loops past the last step, the regime that extends furthest. We did not test stopping rules on text, where extra loops did not help.

The chain task meets all three conditions once the model is trained for them; web text at these sizes showed no sign of headroom, and its one extendable regime gained nothing from running longer. Two follow-ups we did not run would test this reading: a serial task whose input length does not grow with the number of steps, which would also decide whether a written chain of thought extends when the prompt does not get longer; and training on text that needs more serial steps, such as mathematics and code, compared at equal compute.

The value of a loop is not a constant of the architecture. Even where the loop does not iterate, it buys something on text, and what it buys depends on size, data and the dense model it is compared with: worth at most its own parameters at 10–20M with up to 20 tokens per parameter, about 1.3–1.5 times as many at 40M–160M with 20–40, and at 40M about half of that is what a deeper dense model of the same size also gets. Effective-parameter laws whose loop gain depends on the loop count but not on model size or training tokens describe the larger models well and the smaller ones not at all, and the single pre-registered φ averaged over both. Studies at one scale, or against dense references of one depth, will undersell or oversell the loop depending on where they sit.

Practical guidance. Within the range we measured:

- On text, loop when parameters or data are the constraint, not compute or latency: the advantage at equal parameters grows with model size and tokens per parameter, a dense model was better at equal training compute in every measured comparison but one (level on average over two seeds at the smallest size and the most data, with the lowest learning rate tried for the looped model), and at deployment the loop pays its executed layers in decoding time and KV cache. Try a deeper dense model of the same size first: at 40M it gave about half of the advantage for half the training compute, and at the shortest budget, with its learning rate tuned, all of it (one seed); at 20M, at width 320, it was worse than the 8-layer model.
- Loop a middle block with full backpropagation, start with four loops on text (at 40M–80M eight did about as well at about 1.75 times the training compute), and judge at the target scale: small-scale ablations undersell the loop.
- Tune the looped model's learning rate separately, especially at small scale and on tasks that need serial steps, where at a rate that suits the dense model most looped runs did not learn the chain task in the given steps.
- Do not expect test-time scaling on text from standard training; sweep the loop count at inference to see whether a model can use more loops, and probe it to see whether it computes step by step even when it cannot.
- If a model must decide how long to run, its confidence is a good stopping signal, made reliable in all five seeds by training a few loops past the last step. Within the training range of our task a written chain of thought, supervised on every intermediate value, was the cheaper choice; its failure beyond it came from longer prompts and says little about inputs of fixed length.

8 Limitations, pre-registration and reproducibility

- **Scale, data and evaluation.** Models of 10M–160M parameters with a 16k vocabulary, far below the sizes at which looped language models are deployed, trained on one source (FineWeb-Edu, no document seen twice) and evaluated by validation loss only, without downstream benchmarks. The trends with size may bend; there are five sizes, one seed at 80M and one looped cell at 160M, and single seeds for the random-loop language models and the KV-sharing test. Data-limited or repeated-data regimes, where loops may matter more (Chen et al., 2026b; Gülbahar et al., 2026), are not covered. Seeds change only the initialisation, except in the data-order experiment.
- **Depth on text.** The dense ruler has eight layers at every size; the deeper dense models of S10 cover two sizes with one shape each (18 layers against the looped model's 20 executed layers, narrow at 20M), and their learning rate was checked with one seed, its best at 40M being the highest rate tried; at 80M–160M, where the multiplier is largest, the share due to depth is unmeasured. No dense control of equal executed depth at the looped model's width, which would have more parameters, was trained on text.
- **Equal compute.** 62 of the 320 equal-compute comparisons rest on an extrapolated dense loss, bracketed by two methods, and the measured comparisons are not independent of each other.
- **Learning rates.** The dense rates come from a sweep at one budget; the check of looped cells at 20M tried rates only up to the dense rate, which nine of its ten cells preferred; looped rates were re-tuned for one cell at 10M–40M with one seed (two at 20M), where the shared rate understated the looped model by up to 0.024 nats at 10M and its best rate at $80N$ and $160N$ was the lowest tried; the other cells and 80M–160M use the dense rates. On the synthetic task, ties in the learning-rate criterion went to the lowest rate, several models were best at the lowest rate tried, and extrapolation is sensitive to the rate.
- **One synthetic task family**, chains of table lookups with 6.6M-parameter looped models; extrapolation length varies widely across seeds, and a pointer-chasing control was not learned and was dropped. Probes show where a value can be read out, not that the model uses it. The chain-of-thought comparison covers one task and one recipe; its prompt grows with the number of steps, so whether a written chain extends on a task whose input length does not grow was not tested; the chain-of-thought models were supervised on every intermediate value and got no help with length generalisation, and continuous thoughts (Hao et al., 2025) were not tried.
- **Deployment** was measured on one GPU type, for models of at most 160M parameters, with one KV cache per executed layer; KV sharing was tested only as a change after training, at 20M.
- **Exploratory analyses.** The multiplier, the equal-compute bounds and the probes of the language models were chosen after seeing data. The stopping comparison (S8) was planned after the frozen rule's results were known, though frozen before it ran, and its selection criterion picked the confidence rule at 0.99 that the text highlights only for $r = 8$ and the random loop count, and only once votes are set aside. The readings of P3c and P4 were fixed after the results, and the S10 readings at each model's best learning rate were computed by a script written after the results, from a rule its plan fixed. The pre-registered verdicts stand on their own.

Pre-registration, deviations and reproducibility. The pre-registration was frozen on 2026-09-23 before any training run, and its SHA-256 was committed to the repository the same day; the synthetic design was frozen on 2026-10-09 and the three supplementary plans on 2026-10-13, 2026-10-15 and 2026-10-27, each before its runs. Every later change is dated in an append-only deviation log with its reason and its effect on the conclusions, including the changes made before any data and three reviews (2026-10-14, 2026-10-15 and, of the paper, 2026-10-27) that found and corrected errors, overstatements and incomplete disclosures (Appendix H). Among them, a verdict had been judged with a quantity the frozen synthetic design did not specify, and an evaluation it specified (the state-rule stop of the loops = steps model) was computed but not reported until the second review. Code, configurations, the per-run results and training logs of all 242

language-model runs and 135 synthetic-task runs, the fits, and the scripts that make the fits and the figures are available at `https://github.com/bethehand/looped-scaling`; the remaining tables and numbers are computed from these results. Fits and figures run on a laptop CPU; the model checkpoints (about 560 GB) are not included.

Use of AI assistance. The study was carried out with the help of an AI assistant (Claude, by Anthropic), which drafted code, analyses, documents and this text under the direction of the author. Every decision recorded in the pre-registration and the deviation log, and responsibility for the content, lie with the author. Every number in the paper can be recomputed from the results in the repository.

References

Cem Anil, Ashwini Pokle, Kaiqu Liang, Johannes Treutlein, Yuhuai Wu, Shaojie Bai, Zico Kolter, and Roger Grosse. Path Independent Equilibrium Models Can Better Exploit Test-Time Computation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2211.09961.

Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. Relaxed Recursive Transformers: Effective Parameter Sharing with Layer-wise LoRA. In *International Conference on Learning Representations (ICLR)*, 2025a. arXiv:2410.20672.

Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoun Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation. *arXiv preprint arXiv:2507.10524*, 2025b.

Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep Equilibrium Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. arXiv:1909.01377.

Andrea Banino, Jan Balaguer, and Charles Blundell. PonderNet: Learning to Ponder. In *8th ICML Workshop on Automated Machine Learning*, 2021. arXiv:2107.05407.

Arpit Bansal, Avi Schwarzschild, Eitan Borgnia, Zeyad Emam, Furong Huang, Micah Goldblum, and Tom Goldstein. End-to-end Algorithm Synthesis with Recurrent Networks: Logical Extrapolation Without Overthinking. *arXiv preprint arXiv:2202.05826*, 2022.

Nora Belrose, Igor Ostrovsky, Lev McKinney, Zach Furman, Logan Smith, Danny Halawi, Stella Biderman, and Jacob Steinhardt. Eliciting Latent Predictions from Transformers with the Tuned Lens. *arXiv preprint arXiv:2303.08112*, 2023.

Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, Joshua Lochner, Caleb Fahlgren, Xuan-Son Nguyen, Clémentine Fourrier, Ben Burtenshaw, Hugo Larcher, Haojun Zhao, Cyril Zakka, Mathieu Morlon, Colin Raffel, Leandro von Werra, and Thomas Wolf. SmolLM2: When Smol Goes Big – Data-Centric Training of a Small Language Model. *arXiv preprint arXiv:2502.02737*, 2025.

Tamay Besiroglu, Ege Erdil, Matthew Barnett, and Josh You. Chinchilla Scaling: A replication attempt. *arXiv preprint arXiv:2404.10102*, 2024.

Yanbei Chen, Anirudh Goyal, and Raghuraman Krishnamoorthi. Scaling Laws for Looped Mixture of Experts. *arXiv preprint arXiv:2609.40316*, 2026a.

Zixi Chen, Akshay Vegesna, Samip Dahal, and Andrew Gordon Wilson. How Model Growth, Recursion, and Boundary Operators Influence Scaling Exponents. *arXiv preprint arXiv:2609.19107*, 2026b.

Hanseul Cho, Jaeyoung Cha, Pranjal Awasthi, Srinadh Bhojanapalli, Anupam Gupta, and Chulhee Yun. Position Coupling: Improving Length Generalization of Arithmetic Transformers Using Task Structure. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.20671.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal Transformers. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1807.03819.

Ying Fan, Yilun Du, Kannan Ramchandran, and Kangwook Lee. Looped Transformers for Length Generalization. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2409.15647.

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. *arXiv preprint arXiv:2502.05171*, 2025.

Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training Language Models With Pause Tokens. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.02226.

Alex Graves. Adaptive Computation Time for Recurrent Neural Networks. *arXiv preprint arXiv:1603.08983*, 2016.

Serdar Gülbahar, Lukas Edman, and Alexander Fraser. Squeezing More from Limited Data with Recursive Transformers. *arXiv preprint arXiv:2608.26973*, 2026.

Alexander Hägele, Elie Bakouch, Atli Kosson, Loubna Ben Allal, Leandro von Werra, and Martin Jaggi. Scaling Laws and Compute-Optimal Training Beyond Fixed Training Durations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.18392.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training Large Language Models to Reason in a Continuous Latent Space. In *Conference on Language Modeling (COLM)*, 2025. arXiv:2412.06769.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. Training Compute-Optimal Large Language Models. *arXiv preprint arXiv:2203.15556*, 2022.

Benhao Huang, Chufan Shi, Junlin Chen, Shicheng Wen, Zhengzhong Liu, Eric Xing, and Xuezhe Ma. Towards Looped Models Done Right, Part II: Rethinking at Fixed Points. *arXiv preprint arXiv:2610.06833*, 2026.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361*, 2020.

Harsh Kohli, Srinivasan Parthasarathy, Huan Sun, and Yuekun Yao. Loop, Think, & Generalize: Implicit Reasoning in Recurrent-Depth Transformers. In *Conference on Language Modeling (COLM)*, 2026. arXiv:2604.07822.

Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. *arXiv preprint arXiv:1909.11942*, 2019.

Ryan Lee, Jacob Biloki, Edward J. Hu, and Jonathan May. Sparse Layers are Critical to Scaling Looped Language Models. *arXiv preprint arXiv:2605.09165*, 2026.

Bingbin Liu, Jordan T. Ash, Surbhi Goel, Akshay Krishnamurthy, and Cyril Zhang. Transformers Learn Shortcuts to Automata. *arXiv preprint arXiv:2210.10749*, 2022.

Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas Chandra. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.14905.

Joe Logan. Per-Token Fixed-Point Convergence in Depth-Recurrent Transformers. *arXiv preprint arXiv:2607.14427*, 2026.

Andrei Marchenko, Viacheslav Bezrukov, Oleg Kashurin, Inessa Fedorova, Dmitry Bocharov, Yuliana Shakhvalieva, Maria Tikhonova, and Valerii Ternovskii. Closing the Loop: Practical Training Recipes for Looped Language Models. *arXiv preprint arXiv:2610.00673*, 2026.

Sean McLeish, Arpit Bansal, Alex Stein, Neel Jain, John Kirchenbauer, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, Jonas Geiping, Avi Schwarzschild, and Tom Goldstein. Transformers Can Do Arithmetic with the Right Embeddings. *arXiv preprint arXiv:2405.17399*, 2024.

William Merrill and Ashish Sabharwal. The Expressive Power of Transformers with Chain of Thought. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.07923.

Giovanni Monea, Keshav Ramji, Yousef El-Kurdi, Luis A. Lastras, Yoav Artzi, Nathan Godey, and Ramón Fernandez Astudillo. The Surprising Effectiveness of Shared Memory in Looped Transformers. *arXiv preprint arXiv:2610.02383*, 2026.

nostalgebraist. interpreting GPT: the logit lens. LessWrong blog post, 2020. URL `https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens`.

Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show Your Work: Scratchpads for Intermediate Computation with Language Models. *arXiv preprint arXiv:2112.00114*, 2021.

Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. *arXiv preprint arXiv:2406.17557*, 2024.

Jacob Pfau, William Merrill, and Samuel R. Bowman. Let's Think Dot by Dot: Hidden Computation in Transformer Language Models. *arXiv preprint arXiv:2404.15758*, 2024.

Tomer Porian, Mitchell Wortsman, Jenia Jitsev, Ludwig Schmidt, and Yair Carmon. Resolving Discrepancies in Compute-Optimal Scaling of Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2406.19146.

Hayden Prairie, Zachary Novack, Taylor Berg-Kirkpatrick, and Daniel Y. Fu. Parcae: Scaling Laws For Stable Looped Language Models. *arXiv preprint arXiv:2604.12946*, 2026.

Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with Latent Thoughts: On the Power of Looped Transformers. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2502.17416.

Avi Schwarzschild, Eitan Borgnia, Arjun Gupta, Furong Huang, Uzi Vishkin, Micah Goldblum, and Tom Goldstein. Can You Learn an Algorithm? Generalizing from Easy to Hard Problems with Recurrent Networks. *arXiv preprint arXiv:2106.04537*, 2021.

Kristian Schwethelm, Daniel Rueckert, and Georgios Kaissis. How Much Is One Recurrence Worth? Iso-Depth Scaling Laws for Looped Language Models. *arXiv preprint arXiv:2604.21106*, 2026.

Daria Soboleva, Faisal Al-Khateeb, Robert Myers, Jacob R. Steeves, Joel Hestness, and Nolan Dey. SlimPajama: A 627B token cleaned and deduplicated version of RedPajama. Cerebras blog post, 2023. URL `https://huggingface.co/datasets/cerebras/SlimPajama-627B`.

Shaowen Wang, Bingrui Li, Tansheng Zhu, Ge Zhang, Wenhao Huang, Shen Yan, and Jian Li. Loop-μP: Residual Scaling and Hyperparameter Transfer in Looped Transformers. *arXiv preprint arXiv:2606.18524*, 2026a.

Shaowen Wang, Ge Zhang, Kairong Luo, Yuhao Wu, Shaofan Liu, Jiaheng Liu, Wenhao Huang, Shen Yan, and Jian Li. SMELT: Scaling Laws for Compute-Matched MoE Looped Transformers. *arXiv preprint arXiv:2609.01343*, 2026b.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *arXiv preprint arXiv:2201.11903*, 2022.

Bohong Wu, Mengzhao Chen, Xiang Luo, Shen Yan, Qifan Yu, Fan Xia, Tianqi Zhang, Hongrui Zhan, Zheng Zhong, Xun Zhou, Siyuan Qiao, and Xingyan Bin. Parallel Loop Transformer for Efficient Test-Time Computation Scaling. *arXiv preprint arXiv:2510.24824*, 2025.

Qifan Yu, Zhenyu He, Sijie Li, Xun Zhou, Jun Zhang, Jingjing Xu, and Di He. Enhancing Auto-regressive Chain-of-Thought through Loop-Aligned Reasoning. *arXiv preprint arXiv:2502.08482*, 2025.

Tong Zhang, Junhao Hu, Yun Peng, and Tao Xie. When Does Recurrence Become an Algorithm? Convergence Selection in Weight-Tied Looped Transformers. *arXiv preprint arXiv:2607.20594*, 2026.

Yongchao Zhou, Uri Alon, Xinyun Chen, Xuezhi Wang, Rishabh Agarwal, and Denny Zhou. Transformers Can Achieve Length Generalization But Not Robustly. *arXiv preprint arXiv:2402.09371*, 2024.

Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, Qiyang Min, Hongzhi Huang, Xun Zhou, Wei Ye, Jiaheng Liu, Jian Yang, Yunfeng Shi, Chenghua Lin, Enduo Zhao, Tianle Cai, Ge Zhang, Wenhao Huang, Yoshua Bengio, and Jason Eshraghian. Scaling Latent Reasoning via Looped Language Models. *arXiv preprint arXiv:2510.25741*, 2025.

A The pre-registered hypotheses and decision rules

The pre-registration (frozen 2026-09-23; an English translation of the binding original is in the repository with the commands that verify its hash) fixed the grid of Section 3.2, the fitting protocol of Section 3.3 and these decision rules. Before any data, three changes were made and dated in the deviation log: the training data moved from sample-100BT to sample-10BT, which removed the $80N$ point of the 160M dense model; the data point moved from the mean of a cooldown's last three evaluations to its final evaluation, the three-point mean becoming a robustness variant (Table 9); and a variant that counts N without the output head was added.

A factor effect counts only if it exceeds both the bootstrap half-width and the spread of φ fitted separately to the three seeds at 20M and 40M. The information criterion of H3 turned out to be computable only as a proxy from the Huber loss, which is not a likelihood; it is not used as evidence (Appendix H). Two rules were added during the grid, after the first spikes (2026-09-26) and after the collapse (2026-09-30), each before the results it governs existed, and are dated in the deviation log: spikes are reported per run with a robustness fit without the runs that have them, and a cooldown result enters no fit if its branch starts after a collapse or ends more than 1.0 nats above the lowest trunk loss before it (four results of two runs: three of a 40M whole stack looped 8 times with truncation, seed 43, which collapsed, and the $10N$ result of a 40M whole stack looped 8 times with full backpropagation, seed 44, after a severe spike).

B Training details

Table 6 lists the sizes and Table 7 the supplementary experiments.

Data. The three further validation sets of 20M tokens each come from a 6B-token sample of SlimPajama, from FineMath (finemath-4plus) and from GitHub code (github-code-clean, `https://huggingface.co/datasets/codeparrot/github-code-clean`).

Table 5: The four pre-registered hypotheses, their frozen decision rules and their verdicts.

Hypothesis	Decision rule for support	Verdict
H1: truncated backpropagation lowers φ	$\varphi_{\text{trunc}} < \varphi_{\text{full}}$ with non-overlapping 95% bootstrap intervals, under both equal-token and equal-FLOP accounting; if only under equal tokens, the effect is attributed to the reallocation of FLOPs.	Not supported. Middle block 0.00 vs 0.13 (predicted direction, intervals overlap); whole stack 0.17 vs 0.00 (opposite direction). The equal-FLOP accounting agrees.
H2: a looped middle block has a higher φ than a looped whole stack	$\varphi_{\text{mid}} > \varphi_{\text{whole}}$ with non-overlapping intervals; raw-loss pairs of similar executed depth (middle block $r = 8$ vs whole stack $r = 4$; middle block $r = 4$ vs whole stack $r = 2$) are reported alongside, and a disagreement is reported as "depends on the measure".	Depends on the measure. φ is mixed; raw losses at similar executed depth (36 against 32 and 20 against 16 layers) favour the middle block in 24 of 24 comparisons, by 0.03–0.19 nats, less at larger sizes.
H3: φ saturates in r	The geometric form has the lower extrapolation error in both directions (10M–40M predicting 80M; 20M–80M predicting 10M) and an information-criterion difference above 10.	Not supported. The geometric form predicts 80M from 10M–40M better (0.0085 vs 0.0096 rmse) and 10M from 20M–80M slightly worse (0.0203 vs 0.0200); the data do not separate the two forms.
H4: the compute-optimal r	Under matched training FLOPs the optimal $r \leq 2$, under matched deployment FLOPs larger; computed from the fitted law, without a test.	First half holds trivially (the dense model, $r = 1$, is optimal under training compute); second half not supported (it is optimal under deployment compute too).

Table 6: Model sizes and training settings on text. Size names give the approximate non-embedding parameters; N also counts the output head (Porian et al., 2024), which is large at small widths with a 16k vocabulary. The looped models of a size have the same unique layers plus a $2w^2$ injection map (under 2% of N). The learning rate is that of the embedding and output layers, frozen after a 33-run sweep; the hidden layers use 1.22 times it. Warm-up is $2N$ tokens.

Size	Width w	Heads	N	Tokens per step	Learning rate	Seeds (dense / looped)
10M	320	5	15.4M	81,920	2.19×10^{-3}	3 / 3
20M	448	7	26.8M	131,072	1.53×10^{-3}	3 / 3
40M	640	10	50.1M	204,800	1.83×10^{-3}	3 / 3
80M	896	14	92.7M	327,680	2.03×10^{-3}	1 / 1
160M	1280	20	179.6M	524,288	1.56×10^{-3}	2 / 2 (middle block, $r = 4$)

Extensions. The extensions continue the 10M middle blocks looped 4 and 8 times (two seeds) to $80N$ and $160N$, a 10M whole stack looped 4 times and a truncated one looped 8 times (one seed) to the same budgets, the 10M dense model to $160N$ and the 20M middle blocks looped 4 and 8 times (two seeds) to $80N$.

Learning rates. A sweep of 33 runs at $10N$ (30 dense runs at five rates per width, a $\beta_2 = 0.99$ group at 20M, and three looped runs at 80M) fixed the rate per width (Table 6); the minima of all five widths lie inside the swept range, and $\beta_2 = 0.95$ beat 0.99 by 0.008 nats. The hidden layers use $\eta_0(8/12)^{-1/2} \approx 1.22\,\eta_0$. A check for looped cells, whose decision rule was written before its runs on 2026-09-25, two days after the pre-registration (20M, ten cells at 0.5, 0.71 and 1 times the dense rate, $10N$) found no better rate by the rule fixed in advance (pooled best multiple 0.97, threshold 0.84), so all looped language models use the dense rates. Nine of the ten cells were best at $1\times$, the top of that range, so higher rates were not excluded. The residual scaling of Section 3.1 is meant to keep the best rate independent of r: we found no downward shift of the best rate in this check, but at 10M with longer training the looped model preferred lower rates (Section 4.2).

FLOP accounting. Training FLOPs count two FLOPs per multiply-add over every executed layer, the attention terms and the output head, plus a backward pass of twice the forward cost for every layer that receives gradients; with truncated backpropagation the backward pass covers the prelude, the coda, the output head and the last $r/2$ loops.

Table 7: The ten supplementary experiments, each run under a plan frozen before its runs (the S8 plan was written after the frozen stopping rule's results on the same checkpoints were known).

	Plan frozen	What it adds	Where
S1	2026-10-13	Learning rates re-tuned for the dense model and a middle block looped 4 times	§4.2, App. D
S2	2026-10-13	Learning rate and learning of the chain task, three seeds	§6.5
S3	2026-10-13	Inference cost	§4.5, App. F
S4	2026-10-15	Dense models continued to $640N$ (10M) and $320N$ (20M)	§4.4, App. D
S5	2026-10-15	Chain of thought and pause tokens on the chain task	§6.2, §6.3
S6	2026-10-15	KV cache shared across loops after training	§4.5, App. F
S7	2026-10-15	Data order	§5, App. C
S8	2026-10-15	Answer-free stopping rules	§6.6, App. G
S9	2026-10-15	Loops = steps with 0–4 random extra loops	§6.6
S10	2026-10-27	Dense models with 18 layers and the looped model's parameters	§4.3, App. E

The grid at equal parameters. Table 8 gives the looped minus dense loss per cell and size, averaged over the three budgets.

Table 8: Looped minus dense loss at equal parameters and tokens, mean over the budgets $10N$, $20N$ and $40N$ and over seeds (negative: looped better). *Includes a seed with a severe loss spike; without it, -0.029.

Cell	10M	20M	40M	80M
Middle block, $r = 2$	$+0.068$	$+0.025$	-0.010	-0.015
Middle block, $r = 4$	$+0.055$	$+0.002$	-0.030	-0.039
Middle block, $r = 4$, truncated	$+0.079$	$+0.031$	$+0.006$	$+0.009$
Middle block, $r = 8$	$+0.031$	-0.003	-0.033	-0.033
Middle block, $r = 8$, truncated	$+0.039$	$+0.001$	-0.022	-0.018
Whole stack, $r = 2$	$+0.172$	$+0.079$	$+0.021$	$+0.003$
Whole stack, $r = 4$	$+0.150$	$+0.085$	$+0.047$	$+0.025$
Whole stack, $r = 4$, truncated	$+0.044$	$+0.003$	-0.014	$+0.017$
Whole stack, $r = 8$	-0.074	-0.055	$+0.213^{*}$	-0.003
Whole stack, $r = 8$, truncated	-0.053	-0.041	-0.042	-0.020

Two observations from the grid. Truncated backpropagation, predicted to hurt, helped whole-stack loops with four loops at every size (Table 8): at 20M a whole stack looped 4 times reached 3.646 with truncation and 3.783 with full backpropagation at $10N$ (three-seed means); with eight loops it was slightly worse at 10M–20M. At 10M the only looped cells better than the dense model on average over the three budgets were whole-stack loops with $r = 8$. Truncation and whole-stack loops also brought instability: 30 of the 111 runs had at least one loss spike, 25 with truncated backpropagation (16 of these whole-stack loops) and five whole-stack loops with $r = 8$ and full backpropagation; one run collapsed. In an exploratory follow-up, a 10M whole stack looped 4 times with truncation, trained at half the learning rate in three seeds, had no spike at all (4, 1 and 10 at the original rate); it was worse by 0.044 nats at $10N$ and better by 0.064 at $40N$, which cannot be credited to the missing spikes alone, since longer budgets also favour a lower constant rate.

C Fits and robustness

The fits use a Huber loss in log space ($\delta = 10^{-3}$), 500 random starts and 1000 block-bootstrap resamples. The primary ruler, fitted to 54 dense results, is $E = 0.050$ (at the lower bound of the fit), $A = 6.98$, $\alpha = 0.054$, $B = 1065$, $\beta = 0.370$; the joint fit over all 352 results gives $E = 1.94$, $A = 61.7$, $\alpha = 0.267$, $B = 197$, $\beta = 0.266$. Under the constant-φ model, setting every φ to 0 raises the Huber loss only from 0.00352 to 0.00379, and setting it to 1 raises it to 0.0142. Table 9 gives the primary φ under the robustness variants of the pre-registration and of the deviation log, each fixed before the results it governs. Fitted

separately to each seed, φ varies by 0.10–0.21 per cell at 20M and 0.00–0.11 at 40M; the factor effects of Table 2 (0.13–0.17) do not exceed the 20M spread.

Table 9: φ (primary fit) under robustness variants, in the order middle block full / middle block truncated / whole stack full / whole stack truncated.

Variant	φ
Primary analysis	0.13 / 0.00 / 0.00 / 0.17
Without the 30 runs with loss spikes	0.13 / 0.11 / 0.00 / 0.22
Mean of the last three evaluations (the pre-registered data point)	0.14 / 0.00 / 0.00 / 0.17
Second validation set (SlimPajama)	0.31 / 0.18 / 0.02 / 0.30
Without $r = 8$	0.07 / 0.00 / 0.00 / 0.07
Half the budget	0.08 / 0.00 / 0.00 / 0.15
N without the output head	0.07 / 0.00 / 0.00 / 0.14
Equal-FLOP accounting	0.13 / 0.08 / 0.00 / 0.16

For H3, the pre-registered extrapolation (10M–40M to 80M and 20M–80M to 10M, with the dense-only 160M size in neither training set) gives a root-mean-square error of 0.0096 and 0.0200 for r^{φ} and 0.0085 and 0.0203 for the geometric form. Five of the seven robustness variants show the same split; without $r = 8$ the split is reversed, and on the second validation set r^{φ} is better in both directions. For H4, the fitted law makes the dense model optimal at every training budget from 10^{17} to 10^{20} FLOPs, with the smallest available r the best looped choice, 0.00–0.17 nats behind; under deployment compute the dense model is optimal as well, with either set of ruler parameters.

Loop-by-loop probes of the language models. The state after each loop k is read in three ways: with the model's own output layers (the logit lens of nostalgebraist, 2020, which here is the same as running the model with k loops), with a learned linear $w \times w$ map in front of the output layers (in the spirit of the tuned lens of Belrose et al., 2023, but trained on the next token rather than distilled from the model's own output), and with a learned linear head; the maps are trained on 10M unseen training tokens and evaluated on 2M tokens of the main validation set.

The multiplier plane. A least-squares plane in $\log_2 M$ over the middle block looped 4 times gives a factor of 1.21 per doubling of model size and 1.14 per doubling of tokens per parameter (1.16 with 8 loops), with classical 95% intervals, which include the plane's misfit, of 1.13–1.30 and 1.05–1.24; by these intervals the product term is indistinguishable from zero. On the plane the loop breaks even ($M = 1$) at about $N = 35$M with 20 tokens per parameter and at about $N = 12$M, below the smallest model, with 80; the measured points agree at 80 (1.04 for the 10M model, 1.05 on the plane) but reach $M = 1$ earlier at 20 (the 20M model is at 0.99, the plane at 0.93). At 10M with 10 tokens per parameter the two dense models measured at that number of tokens are out of order (the 20M one, at 5.7 tokens per parameter, is worse than the 10M one), so $N^{\star}$ cannot be extrapolated either.

Published multipliers. Published equal-parameter results mostly restate to a similar range in comparable settings, and to higher values at larger sizes or with far more tokens per parameter. Where a paper reports a non-looped ladder, we interpolate it in the same way. Parcae's models, trained with a loop count sampled around a mean of 8 and backpropagated through 4 of them, and run with 8, are worth 1.31–1.46 dense parameters per parameter at 140M–770M (Prairie et al., 2026), against a ladder trained on the same tokens per parameter, which understates the value. Ouro's models looped 4 times are worth 1.23 and 1.36 at 134M and 374M (about 150 and 50 tokens per parameter) and more than 1.75 at 778M, beyond its largest non-looped model, but less than their parameters at 53M despite about 380 tokens per parameter, measured by mean benchmark accuracy read off a plot (Zhu et al., 2025, App. D.1). The two 12-layer looped models of Saunshi et al. (2025), about 0.5B parameters looped 2 and 4 times, are worth 1.26 and 1.36, and their shallower blocks looped 3, 4 and 6 times 1.47–1.95, at about 750–1500 tokens per parameter. The law of Schwethelm et al. (2026) ($\varphi = 0.46$, 95% interval 0.41–0.53) implies 1.25, 1.45 and 1.56 for 2, 4 and 8 loops in their iso-depth layouts. These studies differ from ours in size, data and recipe, so the agreement is a

consistency check, not a pooled estimate. They barely sample models below 40M trained on 20 or fewer tokens per parameter, where we find a model with a looped middle block worth at most about as many dense parameters as it has. That the value rises with tokens per parameter at equal parameters is a dependence that effective-parameter laws whose loop gain depends on the loop count (and, for mixtures of experts, on the sparsity) but not on the training tokens (Schwethelm et al., 2026; Chen et al., 2026a) do not capture; it agrees with the recommendation of Prairie et al. (2026) to increase looping and data in tandem, and with the finding of Chen et al. (2026b) that with repeated data the best loop count grows with compute.

Where the gain lands, and what the seeds measure. Per token, the gain on web text is a small, broad shift: about half of the tokens improve, and the gain is not concentrated on hard tokens beyond what two dense seeds differ by *(exploratory)*. On math and code text the gain is larger, on average about 3 and 7 times the gain on the main set, but the seed-to-seed spread on those sets is 10–100 times larger as well. Supplementary experiment S7 *(pre-specified)* showed that the code set is even more sensitive to the data order, which our seeds share, than to the initialisation (the math set is not): at 20M, training the same initialisation on three disjoint parts of the data stream moves the main-set loss by only 0.003–0.011 nats, 0.1–0.5 times the spread between initialisation seeds (the plan had predicted one to three times), but moves the code-set loss by 0.21–0.69 nats, up to 7 times that spread, and changes the sign of the looped-minus-dense gap there. Of the plan's four predictions only this one, a code-set spread above 0.1, held: on the main set the data-order spread was smaller than predicted relative to the initialisation spread (and, for the dense model, below the predicted 0.005–0.02 nats), and the looped-minus-dense gap did not vary less than the losses. Both the initialisation and the documents a run happens to see move its out-of-distribution loss strongly at this size, so we draw no conclusions from differences on those sets.

D Learning rates (S1) and equal compute at the largest budgets (S4)

S1 re-tuned the dense model and the middle block looped 4 times at 10M, 20M and 40M: 0.5 to 1.41 times the dense rate, plus 0.35 times where the best rate lay at the lowest one tried, at $10N$–$40N$ (at 10M also $80N$ and $160N$), with one seed (two at 20M). Table 10 gives the gap between the middle block looped 4 times and the dense model at the shared rate (Δ_1) and with each model at its best rate ($\Delta^\star$), and $h = \Delta_1 - \Delta^\star$, the bias of the shared rate against the looped model. The pre-specified readings: the trend with size holds (at $40N$ the improvement from 10M to 40M is 0.050 against 0.065 at the shared rate, above the threshold of half of it), the trend with data holds and grows (10M from $10N$ to $160N$: 0.141 against 0.119), the equal-compute verdict was unchanged at the time (its 10M comparison at $160N$, extrapolated then, was later measured in S4: -0.011, a looped advantage, and $+0.005$ with a second seed; Section 4.4), and the bias exceeds the 0.01 tolerance at 10M only.

Table 10: S1: looped (middle block, $r = 4$) minus dense loss at the shared rate and at each model's best rate (seed 42; 20M: mean of seeds 42 and 43). †The lowest rate tested.

Size, budget	Δ_1, shared rate	$\Delta^\star$, best rates	h	Best rate, dense / looped
10M, $10N$	+0.082	+0.082	+0.000	1 / 0.71
10M, $20N$	+0.044	+0.026	+0.018	1 / 0.5
10M, $40N$	+0.015	−0.005	+0.020	1 / 0.5
10M, $80N$	−0.010	−0.035	+0.024	1 / 0.35†
10M, $160N$	−0.037	−0.059	+0.022	1 / 0.35†
20M, $10N$	+0.028	+0.028	0.000	1 / 1
20M, $20N$	−0.005	−0.005	0.000	1 / 1
20M, $40N$	−0.031	−0.031	0.000	1 / 1
40M, $10N$	−0.008	−0.008	0.000	1 / 1
40M, $20N$	−0.032	−0.036	+0.004	1 / 0.71
40M, $40N$	−0.050	−0.055	+0.005	0.71 / 0.71

On the trunk checkpoints at $32N$, the range of the main-set loss over 4–16 inference loops is 0.30, 0.36, 1.02, 3.39 and 4.10 nats at 0.35, 0.5, 0.71, 1 and 1.41 times the dense rate at 10M, 0.53–3.42 at 20M and 0.77–4.81

at 40M (0.5 to 1.41 times): lower rates make the iteration closer to convergent, but at every rate the better of 8 and 16 loops is worse than the 4 loops of training, by 0.07–3.0 nats.

Extrapolated equal-compute comparisons. Two extrapolations bracket a dense loss beyond the measured curve: in a backtest that drops the last budgets of every measured dense curve, extending the last segment predicts a dense loss 0.023–0.036 nats too low one doubling out (it flatters the dense model), and a power law fitted to the whole curve one 0.006–0.027 too high (it flatters looping). After S4, 62 equal-compute comparisons still need an extrapolated dense loss: 43 at 40M–160M, and 19 at 10M–20M, almost all for the third seed, whose dense curve ends at $80N$. At 160M and $40N$ the bracket reaches from +0.004 to +0.043.

Equal compute at the largest budgets (S4). Table 11 gives the equal-compute comparisons at the largest looped budgets. The S4 plan had predicted gaps between −0.01 and +0.04, the dense model ahead in most and none below −0.02; the last two held, but four of the nine measured top-budget gaps exceeded +0.04 (up to +0.054).

Table 11: The equal-compute comparisons at the largest looped budgets (S4; looped minus dense loss on the main validation set, means over two seeds; negative: looped better). At equal compute the dense model is trained on 2.2–3.8 times the tokens.

	Training FLOPs	Looped − dense loss	
Looped cell (size, tokens per parameter)	vs dense	equal parameters and tokens	equal compute
Middle block, $r = 4$ (10M, $160N$)	2.2×	−0.033	+0.016
Middle block, $r = 8$ (10M, $160N$)	3.7×	−0.051	+0.034
Middle block, $r = 4$ (20M, $80N$)	2.2×	−0.051	+0.022
Middle block, $r = 8$ (20M, $80N$)	3.8×	−0.064	+0.052
Middle block, $r = 4$, 0.35× rate (10M, $160N$)	2.2×	—	−0.011, +0.005 (seeds)

E Depth or sharing (S10)

The plan was frozen on 2026-10-27, before any of its runs. Dense models with 18 layers and the looped model's number of parameters (Table 12) were trained with the recipe of the grid's runs of the same size: the same data in the same order, batch, warm-up and optimiser settings, one constant-rate trunk with cooldown branches ending at $10N$, $20N$ and $40N$ of that size (in tokens identical to the looped runs), the same evaluation sets, and seeds 42, 43 and 44. The embedding and output layers use the frozen rate of their width (2.19×10^{-3} at width 320, 1.53×10^{-3} at 448) and the hidden layers $(18/12)^{-1/2}$ times it, as the code does for any depth; seed 42 was also trained at 0.71 and 1.41 times that rate. Training FLOPs per token are 205M (dense, 8 layers), 456M (looped) and 239M (dense, 18 layers) at 20M, and 364M, 835M and 406M at 40M. None of the ten runs had a loss spike.

Table 12: S10: dense models with 18 layers (18×320, $N = 28.1$M; 18×448, $N = 51.2$M) against the middle block looped 4 times (8 unique layers, 20 executed; $N = 27.2$M and 51.0M), at the rule's learning rate. Losses and gaps are means over three seeds (the gap per seed in parentheses; positive: looped better); M is read off the 8-layer dense ruler; 95% intervals from resampling seeds. At 20M the plan reports the results descriptively: M_{loop} is near 1 at $10N$ and $20N$, and R is unstable (0.58, 10.1 and −0.40).

Size, budget	Dense, 18 layers	Looped	Gap (seeds)	M_{deep}	M_{loop}	R
20M, $10N$	3.654	3.671	−0.017 (−0.010, −0.016, −0.026)	0.84 [0.76, 0.98]	0.74 [0.67, 0.83]	—
20M, $20N$	3.465	3.456	+0.008 (+0.017, +0.013, −0.005)	0.92 [0.85, 1.01]	0.99 [0.92, 1.11]	—
20M, $40N$	3.335	3.307	+0.028 (+0.037, +0.033, +0.015)	0.94 [0.89, 1.00]	1.16 [1.09, 1.25]	—
40M, $10N$	3.397	3.394	+0.003 (+0.004, +0.004, −0.001)	1.10 [1.04, 1.14]	1.15 [1.08, 1.20]	0.67 [0.43, 1.03]
40M, $20N$	3.232	3.217	+0.015 (+0.017, +0.018, +0.010)	1.11 [1.09, 1.13]	1.26 [1.21, 1.29]	0.46 [0.38, 0.55]
40M, $40N$	3.119	3.090	+0.029 (+0.030, +0.032, +0.026)	1.11 [1.10, 1.12]	1.34 [1.31, 1.36]	0.36 [0.32, 0.41]

Readings fixed in advance. The primary reading is R at 40M averaged over the three budgets: at least 0.75 reads as "most of the advantage is a depth effect", 0.25–0.75 as "partly depth, partly sharing" and below 0.25 as "looping buys more than depth". It is 0.50 (95% interval 0.41–0.62). The plan would also have reported the two models as on par at 40M had the gap been within 0.01 nats at all three budgets; it is not (+0.015 and +0.029 at $20N$ and $40N$).

Learning rate. Seed 42 at 0.71, 1 and 1.41 times the rule's rate: at 40M 3.421, 3.396 and 3.382 at $10N$, 3.246, 3.231 and 3.225 at $20N$, and 3.126, 3.118 and 3.117 at $40N$; at 20M the rule's rate was the best at every budget. Because 1.41 times the rate beat the rule's rate by more than 0.01 nats at 40M and $10N$ (by 0.014), the plan asks for the readings also at the best rate tried, as in S1, the primary reading staying at the rule's rate. With each model at its best tested rate (seed 42; the looped model's best rates, from S1, are 1, 0.71 and 0.71 times the dense rate at $10N$, $20N$ and $40N$; both read off the 8-layer dense runs at their frozen rates, seed means, with `fit/supp3_best_lr.py`), the gap is −0.010, +0.015 and +0.036, M_{deep} 1.37, 1.17 and 1.12, M_{loop} 1.18, 1.33 and 1.41, and R 1.91, 0.56 and 0.34 (mean 0.94; for seed 42 at the rule's rate 0.58, 0.45 and 0.37, mean 0.47). The mean at the best rates falls in the band the plan reads as "most of the advantage is a depth effect"; it is carried by $10N$, where $\log M_{\text{loop}}$ is smallest, and the primary reading stays at the rule's rate. The analysis script written before the results flagged this case but did not compute these readings; they come from a script written afterwards that implements the rule of the plan.

Predictions. The plan predicted R at 40M between 0.25 and 0.75 (held: 0.50); the 18-layer dense model better than the 8-layer one of the same size by 0.01–0.03 nats at 40M (0.007, 0.016 and 0.020 at $10N$, $20N$ and $40N$: held at two of the three budgets; at 20M it is worse by 0.014, 0.008 and 0.003); and the rule's rate within 0.01 nats of the best of the three (held at five of the six budgets).

Equal compute *(exploratory).* Read as in Section 4.4, without extrapolation: the looped model is behind the 18-layer dense model of equal training compute by 0.166 nats at 40M and $10N$ (compute ratio 2.05) and by 0.193 and 0.112 at 20M and $10N$ and $20N$ (ratio 1.91).

F Deployment measurements

Inference was timed on one RTX 4090 (all other GPUs idle) in bf16, with the decoding step in a CUDA graph, one KV-cache slot per executed layer, random weights, a 512-token prompt and 512 generated tokens, at batch 1 and 32. Table 13 gives the ratios at equal loss for all middle-block and whole-stack cells, against the dense model that reaches the same loss at $40N$ (interpolated between dense sizes).

Table 13: S3: looped over dense, at equal loss. *160M lies beyond the largest dense model (extrapolated).

		Weight	KV cache	Decoding time	Peak memory	
Cell	Sizes	memory	per token	batch 1	batch 1	batch 32
Middle block, $r = 2$	20M–80M	0.93–1.03	1.43–1.51	1.45–1.49	0.99–1.15	1.21–1.24
Middle block, $r = 4$	20M–160M	0.81*–0.96	2.20–2.41	2.37–2.47	0.98*–1.20	1.57–1.58
Middle block, $r = 8$	20M–80M	0.83–0.92	4.01–4.23	4.27–4.56	1.08–1.34	2.29–2.40
Whole stack, $r = 4$	20M–80M	1.03–1.11	4.02–4.27	4.00–4.11	1.18–1.84	2.32–2.41
Whole stack, $r = 8$	20M–80M	0.81–0.85	6.96–7.23	7.57–7.91	1.27–1.50	3.38–3.91

For KV sharing (S6), the first 262,144 tokens of the main validation set were decoded token by token in 256 windows of 1024, with one cache slot per executed layer (whose loss equals that of the ordinary forward pass to four decimals) and with one slot per unique layer, written at a position's last loop and read by every later loop. The models are the 20M trunk checkpoints at $32N$ (seed 42); Figure 8b gives the loss differences, and the random-loop model run with 16 loops instead of 8 also shares at no cost.

At batch 1 the larger KV cache cancels the saving in weights: the looped model needs 1–2% less peak memory at 80M–160M and more at 20M–40M (up to 1.2 times). In this range, loops save parameters, not latency or memory; the saving would grow where weights dominate memory, for example one sequence with

a short context in a much larger model, which we did not test. Truncated backpropagation does not make the cache shareable here, unlike in Huang et al. (2026). Among the fixed-loop models, a lower learning rate loses less (0.100 against 0.170 with 4 loops). The S6 plan had predicted that the fixed 8-loop model would break (a loss increase above 0.3) and the truncated one would lose less than 0.1; the increases were 0.09 and 0.14, while the random-loop models shared at no cost and the lower learning rate lost less, as predicted (two of four).

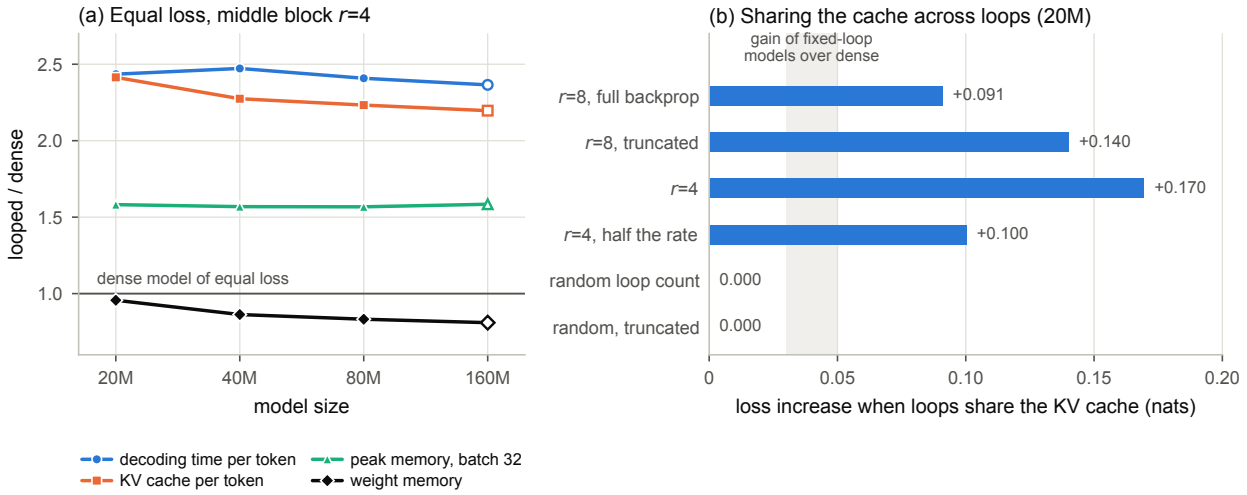


Figure 8: Deployment cost. **(a)** A middle block looped 4 times against the dense model of equal loss at $40N$: ratios of decoding time per token (batch 1), KV cache per token, peak memory at batch 32 (1k context) and weight memory (open markers: 160M lies beyond the largest dense model). **(b)** Sharing the KV cache across loops after training (20M, one cache slot per unique layer, written at a position's last loop): the loss increase on the main set; the shaded band is the fixed-loop models' gain over the dense model of the same size.

G The synthetic task

Data and training. Tokens are the values 0–96 and a few markers (start, separator, lookup, answer and padding); a problem is a start value followed by d steps of the form `LUT k ;`, with the table index k written as one of the value tokens 0–31, and the answer marker. Problems are drawn online. During the first half of training the largest step count grows linearly from 1 to 8; the second half draws uniformly from 1–8. Models have width 256 and attention heads of dimension 64, and the looped models use the input injection and residual scaling of the language models. Training uses AdamW ($\beta = (0.9, 0.95)$, weight decay 0.1, gradient clipping 1.0), 400 warm-up steps and a cosine decay to a tenth of the peak rate, 512 problems per step, 32,000 steps. The two deepest conditions of S5 and S9 accumulated gradients over two half-batches to fit in memory (the same loss and update). Probes are linear maps from the state after each loop (or layer) to the 97 values, trained on 4000 problems with 8 steps and tested on 1000, at the table-index, separator and next-operation positions of each step, keeping the best.

Learning rates. The sweep (seed 1; 2.5×10^{-4}, 5×10^{-4} and 10^{-3}, extended at an edge) chose by mean in-range accuracy at the condition's own loop count: dense 8 and 20 layers 10^{-3}, dense 36 layers 5×10^{-4}, looped $r = 2$ and $r = 8$ 2.5×10^{-4}, looped $r = 4$, random loop count, loops = steps, the S9 model and the chain-of-thought models 1.25×10^{-4}, the pause model 5×10^{-4}. These are the rates of the embedding and output layers; the hidden layers use $(L/12)^{-1/2}$ times them, with L the number of unique layers (1.22, 0.77 and 0.58 for 8, 20 and 36), so the hidden layers of the 36-layer dense model train at about the rate of the 2- and 8-loop models. The criterion saturates at low rates (the loops = steps model scored 0.9999 at three rates and the chain-of-thought models 1.000 at all four), such ties went to the lowest rate by implementation, a rule the protocol had not specified, and the 4-loop, random-loop and S9 models were best at the lowest rate

tried. Extrapolation is sensitive to this choice: in the sweep, the accuracy of the loops = steps model on 9 to 14 steps fell from 1.00 to 0.91 at 2.5×10^{-4} but to 0.05 at 1.25×10^{-4}.

Probes. The *frontier* is the first loop at which each value becomes decodable (probe accuracy ≥ 0.5), and *persistence* is the lowest probe accuracy a value reaches in the loops after that, averaged over the values (Figure 7). In 5 of the 10 seeds of the 20- and 36-layer dense models no intermediate value after the first appears at the step positions: four show only x_1 (probe accuracy 0.50–0.74) and, at the last layer, the answer, and one shows nothing. Measured by the share of values that first become decodable at their own loop, a reading of the design's diagonal share fixed after the results, the loops = steps and 8-loop models are diagonal (0.95 and 0.975) and the others are not (0.125); read as the share decoded most accurately at their own loop, they are 0.625 and 0.425.

S2. Its plan identified the conditions with 4 and 8 loops, the random loop count and loops = steps as extrapolating and left out $r = 2$ on the strength of a number corrected later (Appendix H). Probes of all sweep checkpoints *(exploratory)* show the step-by-step frontier at rates up to 5×10^{-4} and not above, for 4 and 8 loops and the random loop count alike. At 10^{-3} the runs that did not learn the training range reached 0.49–0.98 in range; at 5×10^{-4} the runs that learned it extended about as far as the formal runs (8 loops: 2.7 steps beyond the range against 3.2; loops = steps: 4.0 against 4.8), except the random loop count (2 against 4.2).

Where the chain of thought breaks. As soon as the prompt holds more than 8 tables, the first wrong token of the 8-layer chain-of-thought model is one of the first three written ones in 97–98% of the problems, and even when the correct earlier tokens are given, the tokens after the first two are mostly wrong (accuracy 0.04–0.10) *(exploratory)*.

Stopping rules. Figure 9 shows the confidence rule against the chain length. The plan's selection rule, the highest accuracy over 9–16 steps on the selection set among rules that keep the in-range accuracy within 0.01 of the training loop count's, also admitted votes over loop counts and picked a vote for three of the five conditions; without votes it picks the 0.5% state rule for $r = 4$, the confidence rule at 0.99 for $r = 8$ and the random loop count, and at 0.9 for $r = 2$; for S9 it picks the 0.5% state rule with or without votes. Where it picked a vote, the vote runs all 32 loops and comes within 0.013 of the best fixed count without beating it.

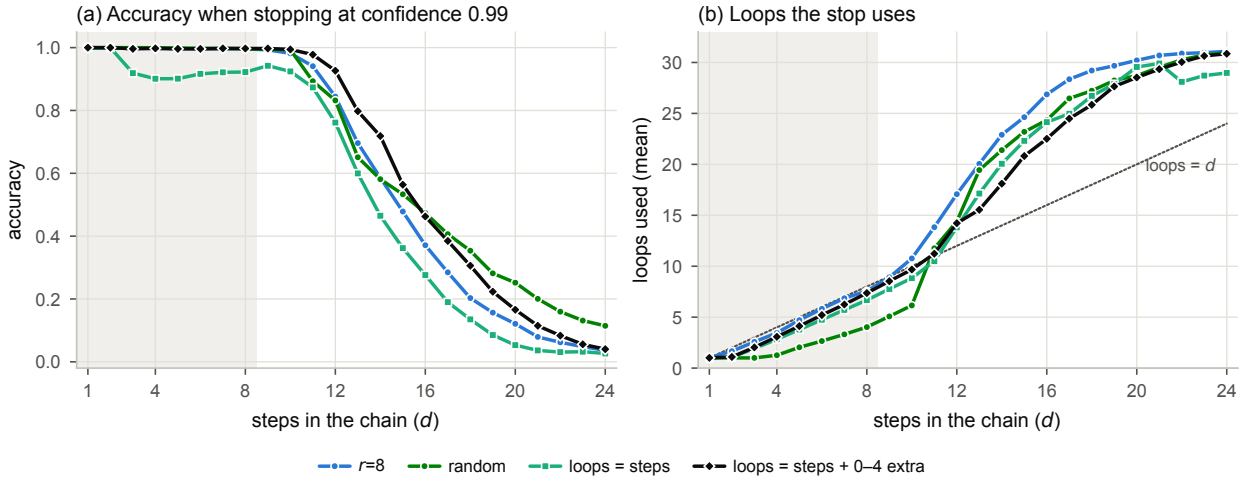


Figure 9: Stopping when the model's top probability reaches 0.99 (reporting set, mean over five seeds). **(a)** Accuracy. **(b)** Loops used: the stop runs few loops on short chains and many on long ones. The model trained with loops = steps loses part of the training range; trained with a few random extra loops, it keeps the training range and stops best of the four beyond it, up to 15 steps.

The pre-specified predictions. Table 14 lists the verdicts.

Table 14: The predictions of the frozen synthetic design and their verdicts.

Prediction	Verdict
P1: the largest solvable length rises from dense 8 layers to $r = 2, 4, 8$; $r = 8$ reaches 16	Holds in direction (4.2, 6.4, 8.0, 9.0); $r = 8$ does not reach 16.
P2: dense models of equal depth are not worse, and the gap is at most a quarter of the dense-8 to looped gap	Not supported as stated: they solve 1–1.6 fewer steps, within the tolerance for $r = 8$, not for $r = 4$.
P3a: fixed-loop models lose accuracy sharply at other loop counts	Half right: with fewer loops accuracy drops (the 8-loop model keeps 0.67 in range at 4 loops); with more loops the 4- and 8-loop models do not, in the direction that favours looping (with 16 loops the 8-loop model keeps 0.997 in range and solves 11.0 steps instead of 9).
P3b: the random-loop model does not improve with more loops beyond 8 steps	Fails: it gains on long chains (0.08 to 0.49 at 16 steps).
P3c: the loops = steps model with d loops beats 8 loops beyond 8 steps, to at least 16	Holds (by 0.34–0.95 at every length from 10 to 16; the start at 10 steps and the 0.1 margin were fixed after the results, and at 9 steps the gain is 0.03).
P4: only the loops = steps model's probe table is diagonal	Half right: diagonal shares 0.95 (loops = steps) and 0.975 (8 loops), 0.125 for the others, by a definition fixed after the results; read as the loop at which a value is decoded most accurately, they are 0.625 and 0.425, and P4 would hold.
P5: the gaps are much larger than on text	Holds.

Supplementary predictions. S2: a high learning rate mainly prevents learning (Section 6.5). S5 held one of three: the 8-layer chain-of-thought model was predicted to extend at least as far as the looped models (≥ 0.7 at 12 steps) and reached 0.014; pause tokens were predicted to add less than 0.1 and added nothing; the 36-layer chain-of-thought model was predicted to reach 0.8 at 12 steps and reached 0.017. S8 held two of five (Table 15): the prediction rule stayed within 1.5 times the own loop count in range, and votes did not beat the best fixed count; the prediction rule neither beat the state rule nor recovered 85% of the gain, and the confidence rule brought the loops = steps model to 0.76, above the predicted ceiling of 0.2. S9 held three of four; the one that failed is the frozen state rule's stop (0.52 at 12 steps against a predicted 0.6).

Table 15: S8, all rule families on the reporting set: accuracy at 12 / 16 steps (mean loops used at 12 steps). The rule the plan's selection criterion picks is in the last column.

Model	Own loops	Best fixed	State < 0.02	Prediction unchanged	Confidence ≥ 0.99	Selected
$r = 2$	0.05 / 0.04	0.62 / 0.33	0.45 / 0.20 (29)	0.56 / 0.21 (6.3)	0.70 / 0.30 (8.3)	confidence ≥ 0.9
$r = 4$	0.04 / 0.02	0.83 / 0.29	0.65 / 0.21 (11)	0.48 / 0.09 (6.7)	0.81 / 0.26 (9.2)	vote, loops 8–32
$r = 8$	0.04 / 0.03	0.86 / 0.37	0.71 / 0.23 (17)	0.09 / 0.04 (5.5)	0.84 / 0.37 (17)	vote, loops 8–32
Random	0.77 / 0.08	0.83 / 0.48	0.73 / 0.37 (11)	0.28 / 0.11 (5.0)	0.83 / 0.47 (14)	vote, loops 8–32
Loops = steps	0.96 / 0.36	0.96 / 0.36	0.03 / 0.03 (25)	0.21 / 0.06 (8.8)	0.76 / 0.28 (14)	none eligible

Pointer chasing. The design's second task, following a random function given in the prompt for k hops, was not learned by any of the four conditions tried on it under the recipe calibrated on the chain task: the 36-layer dense and 8-loop models, two seeds each, reached 0.10–0.24 accuracy when trained to the end, and the random-loop and loops = steps models were stopped at 0.13 training accuracy; single-hop retrieval was not learned in that format either. It was dropped before any comparison on it.

H Deviations and corrections

Every item is dated in the deviation log in the repository. Among the corrections, the reviews found a verdict judged with a quantity the frozen design did not specify, a misread synthetic prediction (P3a) and two readings fixed after the results that the paper had not marked (P3c, P4).

- Before any data, the pre-registered training data (sample-100BT) were replaced by sample-10BT, which removed the $80N$ point of the 160M dense model, and the pre-registered data point, the mean of a cooldown's last three evaluations, by its final evaluation (Appendix A).
- Two power failures of the training machine cut CPU speed for a time; runs resumed from checkpoints and no result was affected.
- The synthetic task changed twice before its formal runs, both times for learnability and before any comparison between conditions: a uniform mixture of 1–8-step problems was not learned (a curriculum was added), and modular arithmetic chains were learned too slowly (fixed random lookup tables replaced them). Training was lengthened to 32,000 steps for every condition, three seeds were added to the first two (declared before their results), and the probe was widened from one to three positions per step after a diagnostic showed values stored at other positions.
- The learning-rate criterion of the synthetic task saturated, and ties went to the lowest rate by implementation (Appendix G).
- Three reviews corrected the results and the paper. The first (2026-10-14) found five errors: a verdict on the random-loop model judged by the longest solvable chain instead of the accuracy difference the design specified, a double-counted KV cache in the S3 memory figures, an information criterion for H3 that was only a Huber-loss proxy (no longer cited), an extrapolation test of H3 that had kept 160M in its training set, and a design statement that problems never repeat, false for short ones. The second (2026-10-15) corrected about 35 numbers and statements, among them the verdicts on P3c and P4, the equal-compute claim (now limited to the measured range), a statement that the loops = steps model overwrites its values (it keeps them within its d loops), a 12-step comparison with dense models that had mixed in extra loops and a best over loop counts that had left out some counts; it also reported an evaluation the frozen design specified, the state-rule stop of the loops = steps model, which had been computed but not reported. The third (2026-10-27), of the paper itself, corrected the S8 count (two of five predictions held), two rounding errors, the claim that confidence is the best stopping rule in every condition, the count of voided results, the P3a verdict and the chain of thought's breaking point, added the pre-data changes to the pre-registration and the outcomes of the S4, S6 and S7 predictions, and marked the readings fixed after the results (P3c, P4). Every item is dated in the log.
- Two synthetic conditions of S5 and S9 ran out of GPU memory at the planned batch, having been tested only on a CPU before their plan was frozen; they were rerun with two accumulated half-batches.
- S10 (plan frozen 2026-10-27): its analysis script flagged that the learning-rate rule applied at 40M and $10N$ but did not compute the readings at the best rate; a script written after the results computes them (Appendix E). The script's intervals for the paired gap are not resampled, so the per-seed gaps are reported instead. The checkpoint size stated earlier (485 GB, measured on 2026-10-15) was not updated as S4–S10 added checkpoints; it is about 560 GB.
- One unexplained anomaly: at 160M one looped seed is 0.2–0.6 nats worse than the other on the out-of-distribution sets at $40N$, while their main-set losses are identical.